

DATA WITH BARAA

Indexes

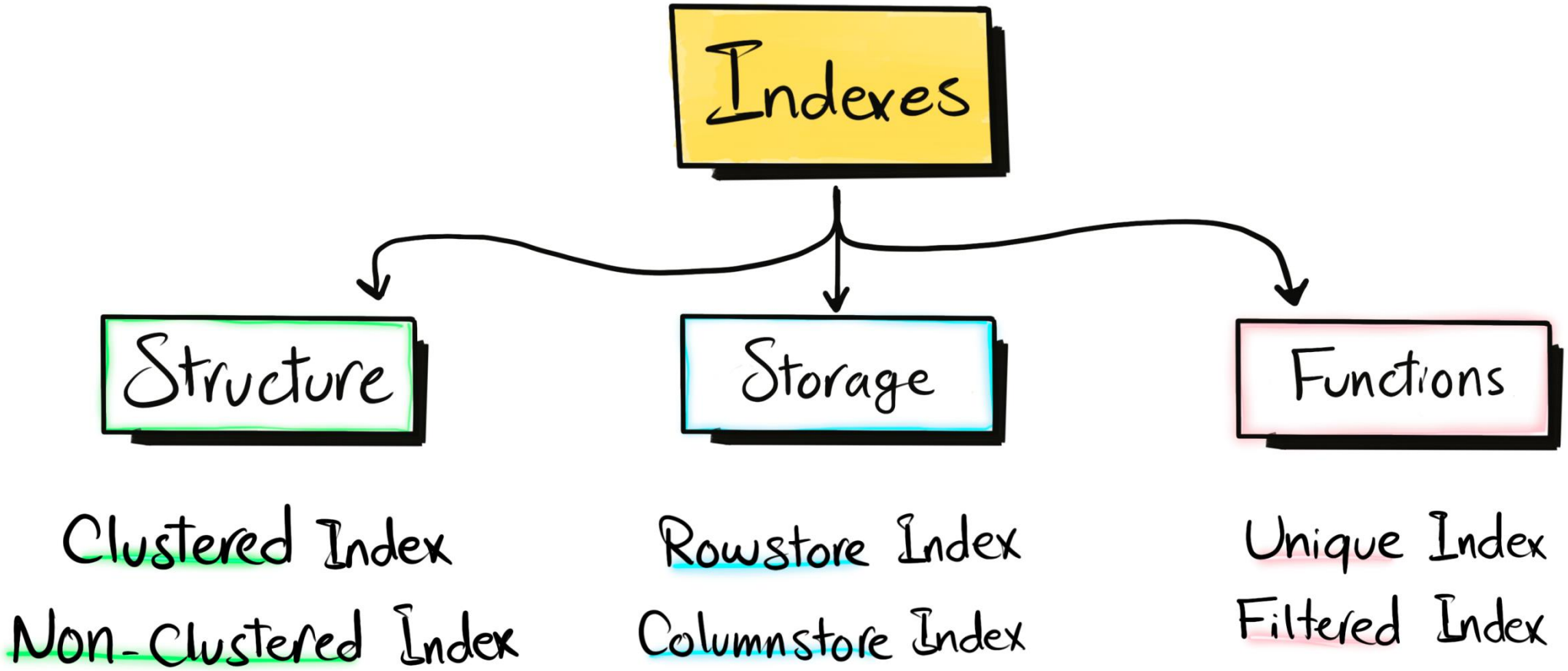
Baraa Khatib Salkini
DATA WITH BARAA
SQL Course | Indexes

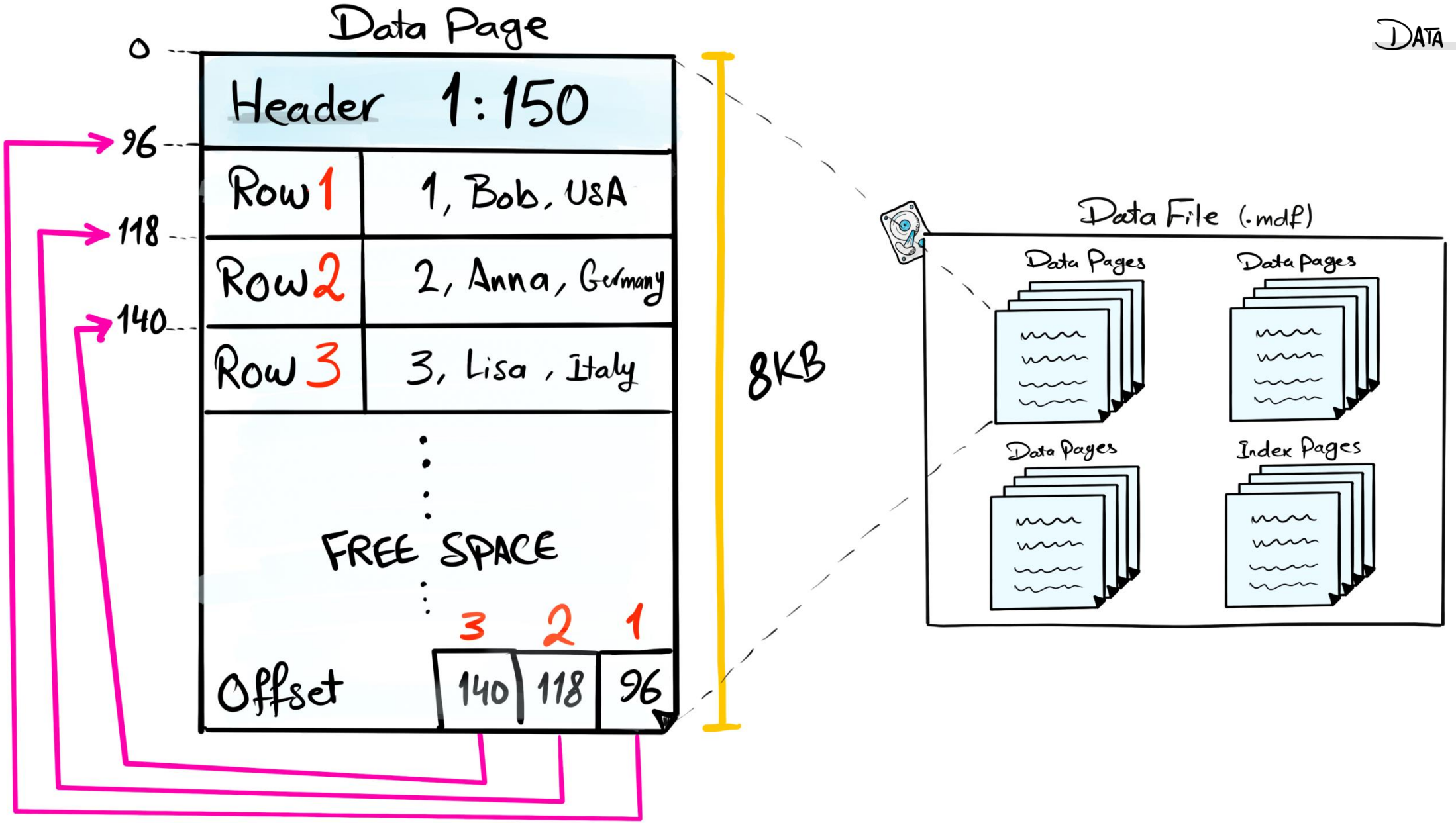


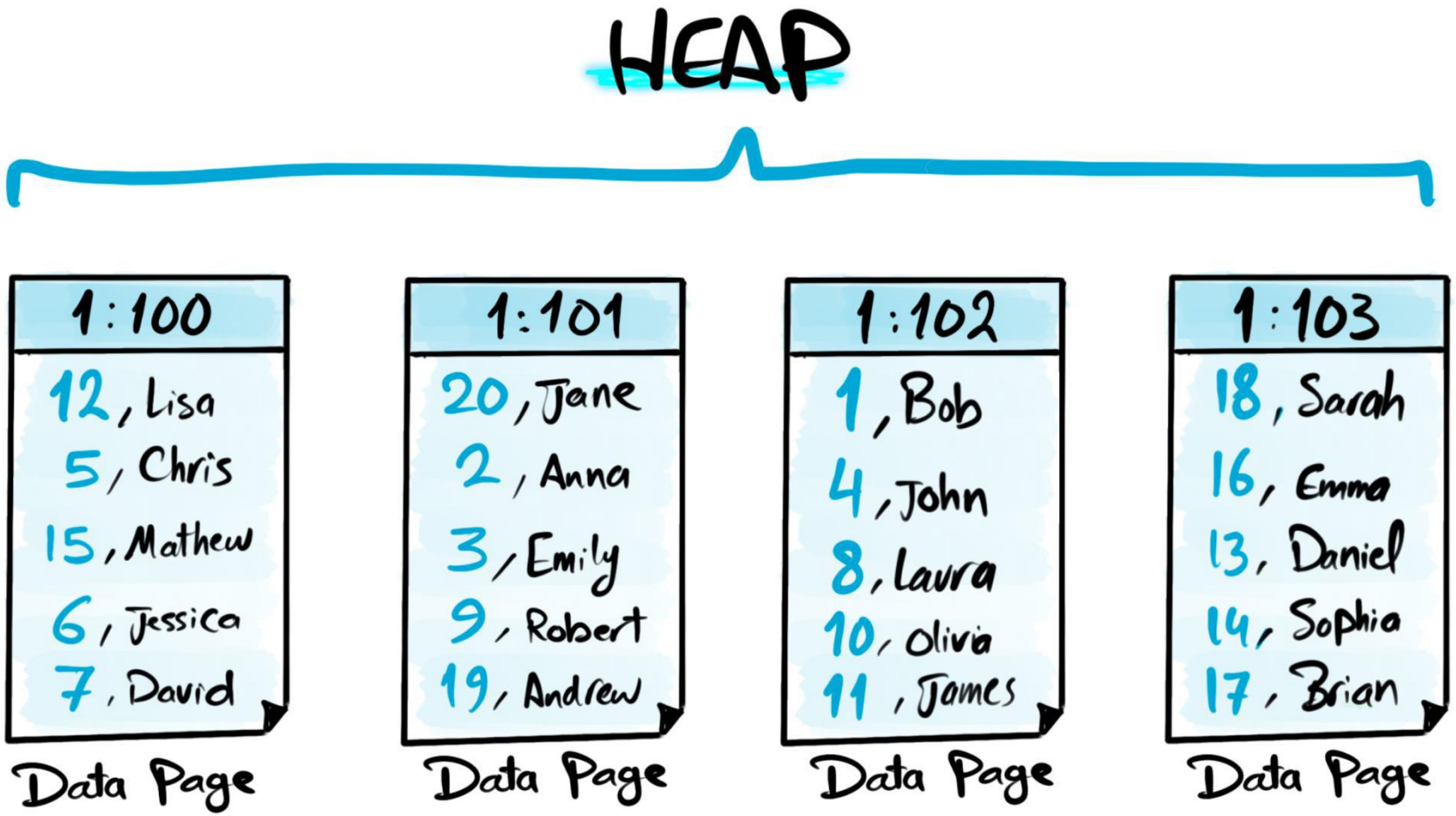
INDEX

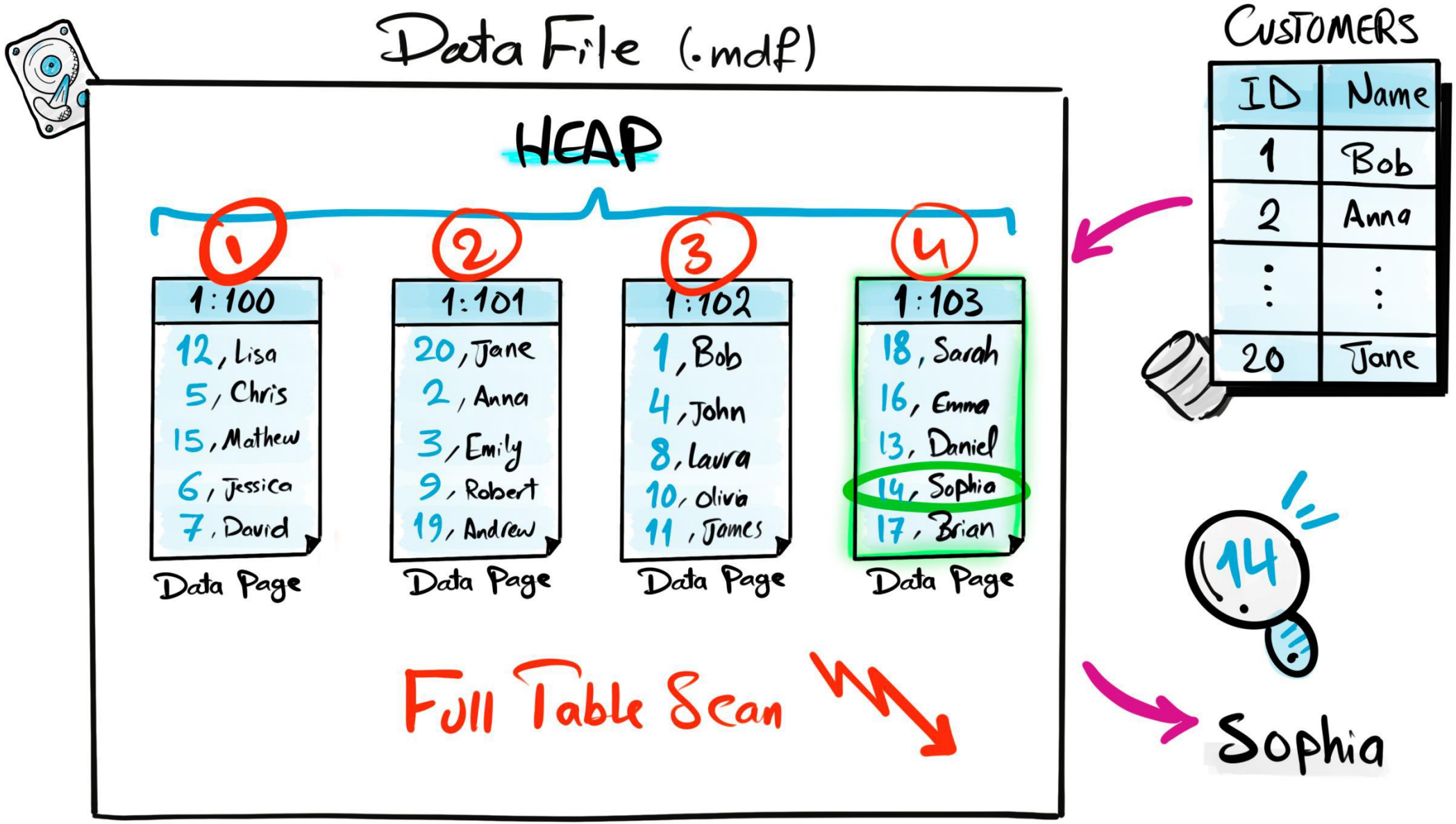
Data structure provides quick access to data,

optimizing the speed of your queries.


Indexes
Structure
Storage
Functions
Clustered Index
Non-Clustered Index
Rowstore Index
Columnstore Index
Unique Index
Filtered Index


Data Page
0
96
118
140
Header 1:150
Row 1
1, Bob, USA
Row 2
2, Anna, Germany
Row 3
3, Lisa , Italy
FREE SPACE
3
2
1
Offset
140
118
96
8KB
Data File (.mdf)
Data Pages
Data Pages
Data Pages
Index Pages


HEAP
1:100
12, Lisa
5, Chris
15, Mathew
6, Jessica
7, David
Data Page
1:101
20, Jane
2, Anna
3, Emily
9, Robert
19, Andrew
Data Page
1:102
1, Bob
4, John
8, Laura
10, Olivia
11, James
Data Page
1:103
18, Sarah
16, Emma
13, Daniel
14, Sophia
17, Brian
Data Page


Data File (.mdf)
HEAP
1
2
3
4
1:100
12, Lisa
5, Chris
15, Mathew
6, Jessica
7, David
Data Page
1:101
20, Jane
2, Anna
3, Emily
9, Robert
19, Andrew
Data Page
1:102
1, Bob
4, John
8, Laura
10, Olivia
11, James
Data Page
1:103
18, Sarah
16, Emma
13, Daniel
14, Sophia
17, Brian
Data Page
Full Table Scan
CUSTOMERS
ID
Name
1
Bob
2
Anna
20
Jane
14
Sophia

B-TREE

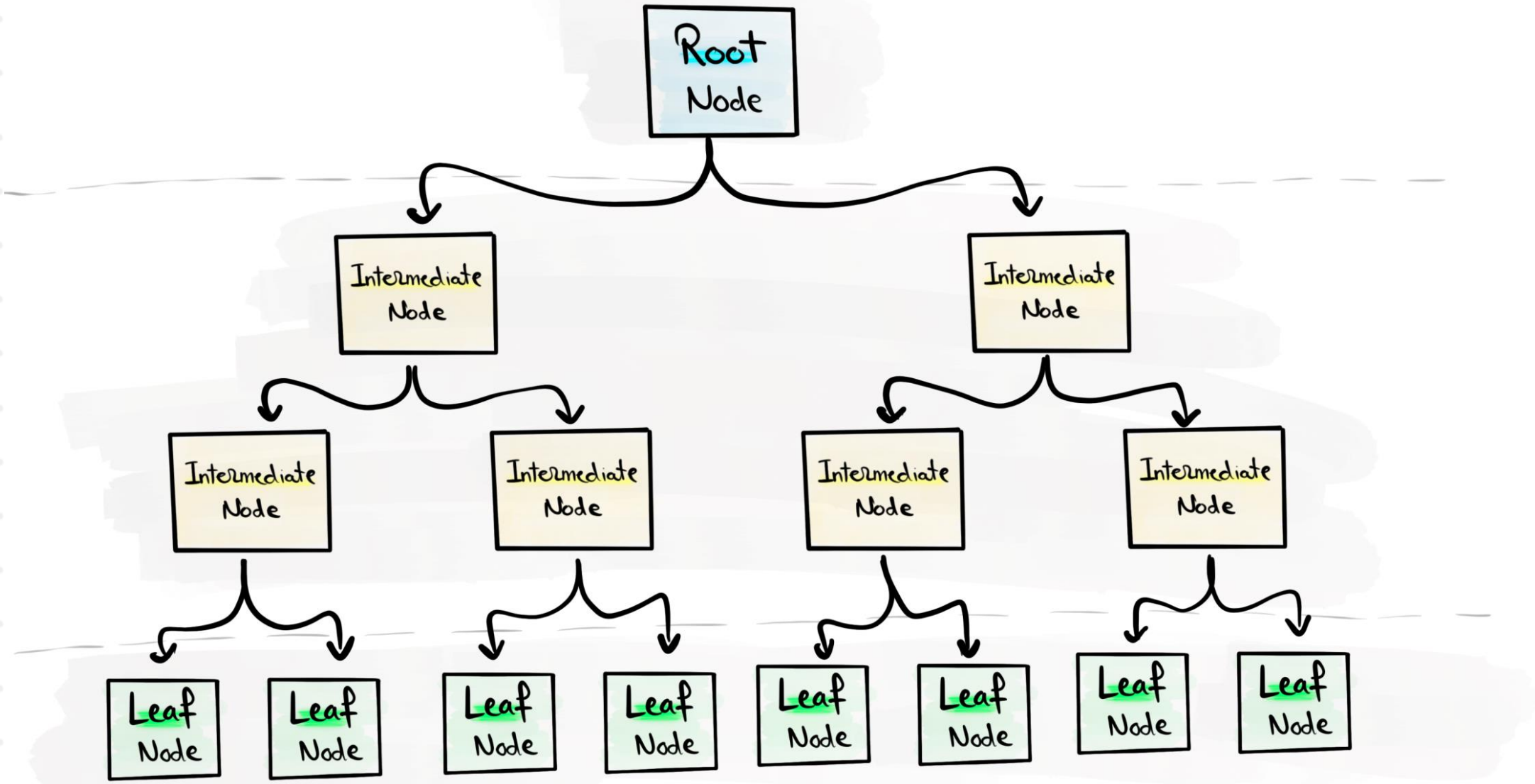
(BALANCE TREE)

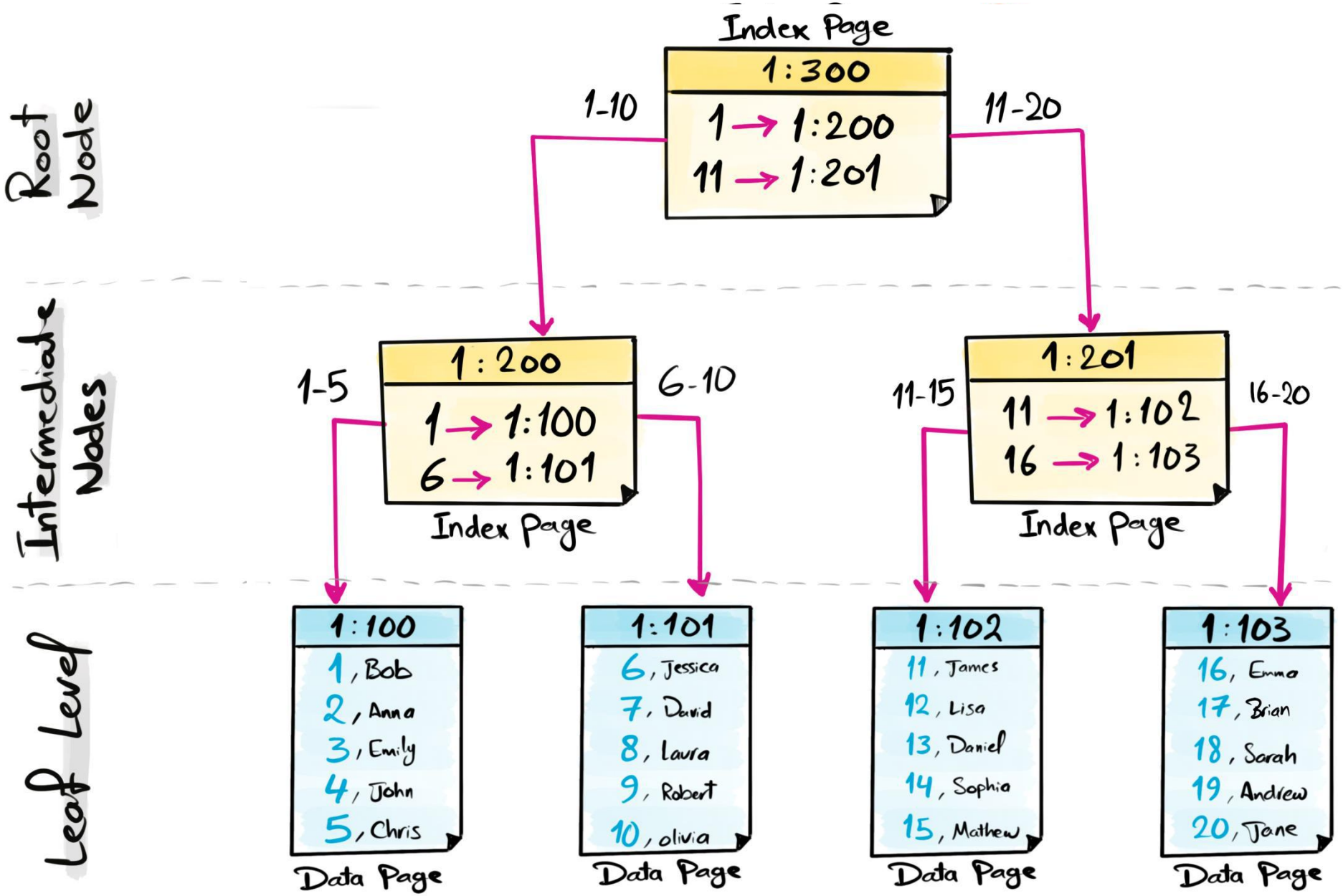
Hierarchical structure storing data at leaves,

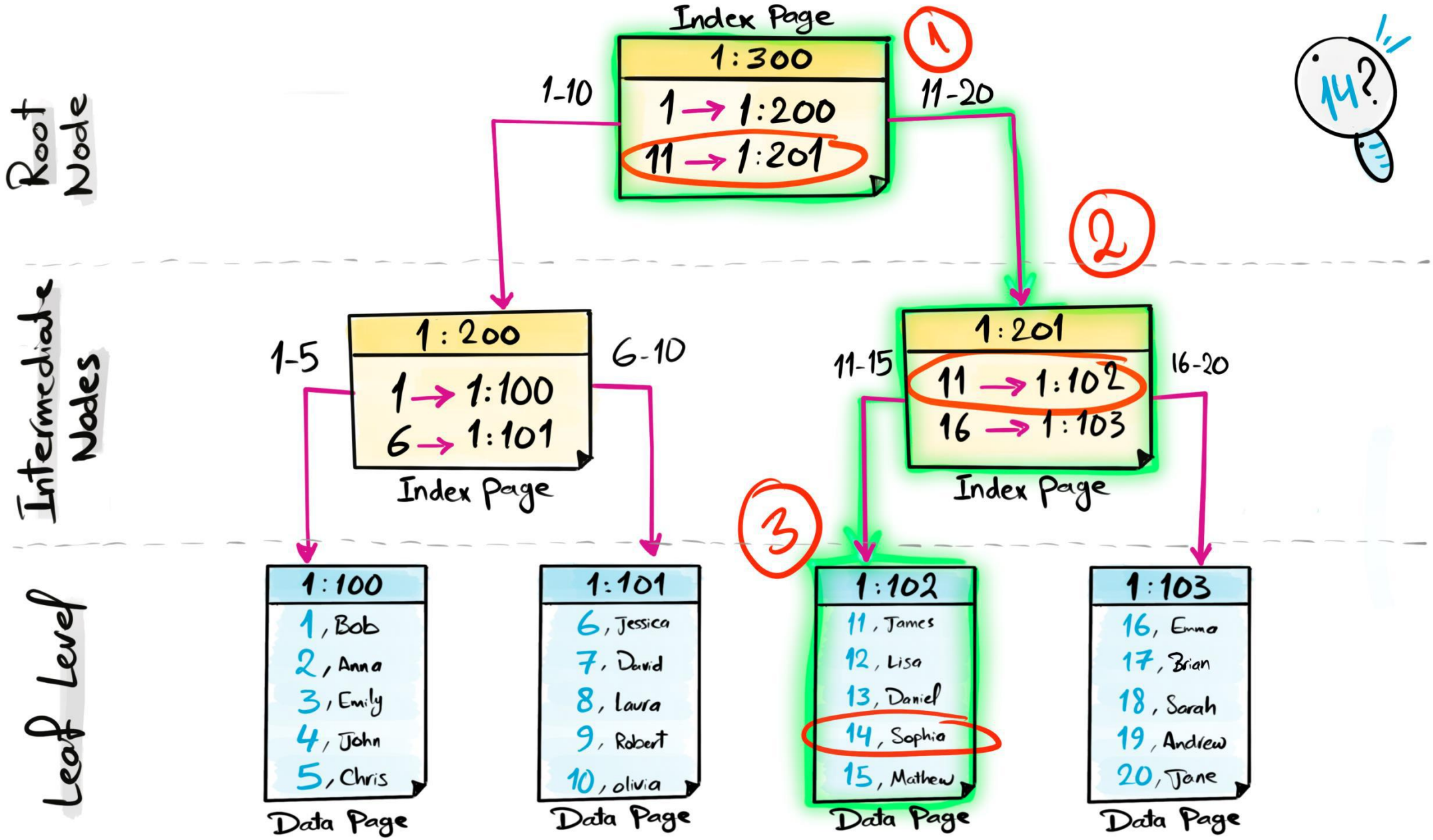
to help quickly locate data.

B-TREE

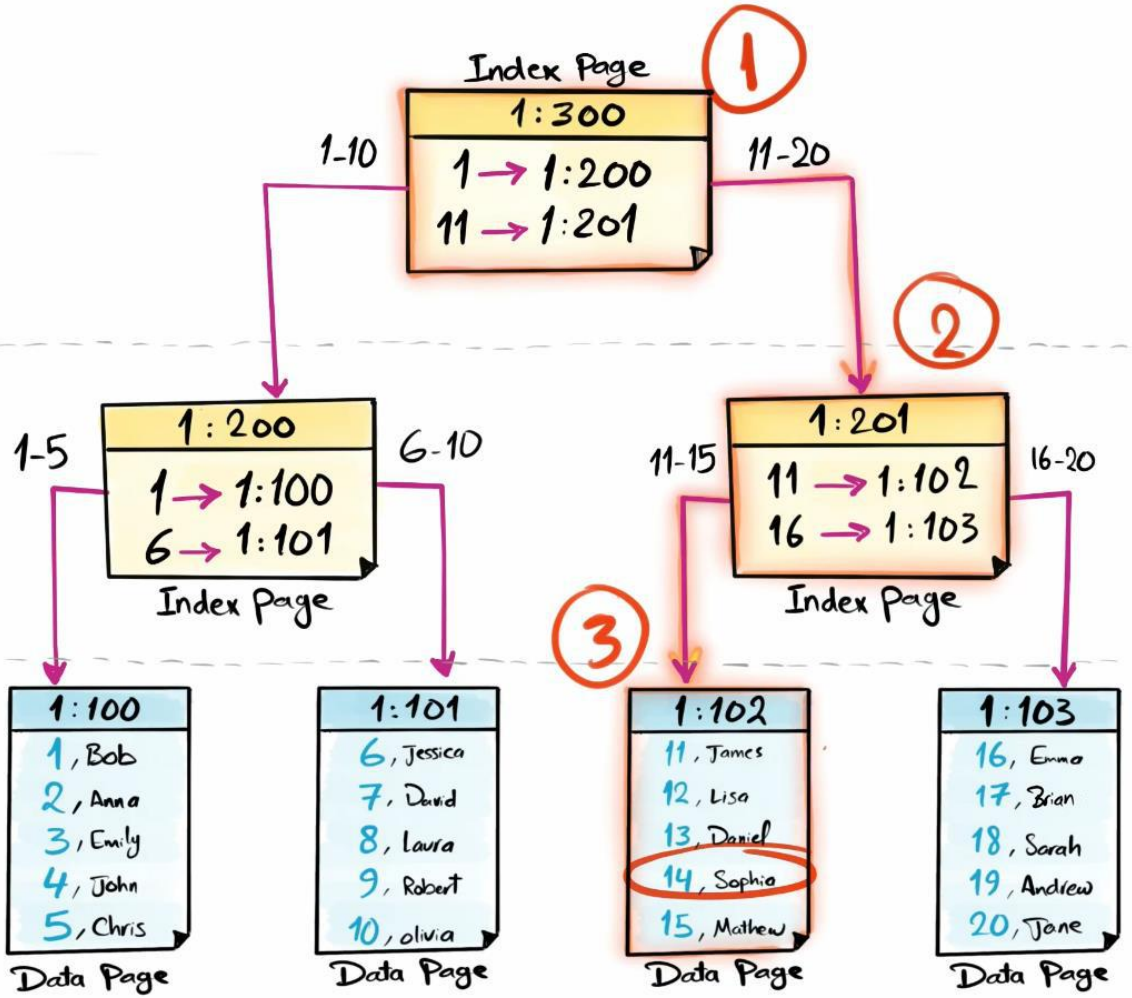
DATA WITH BARAA



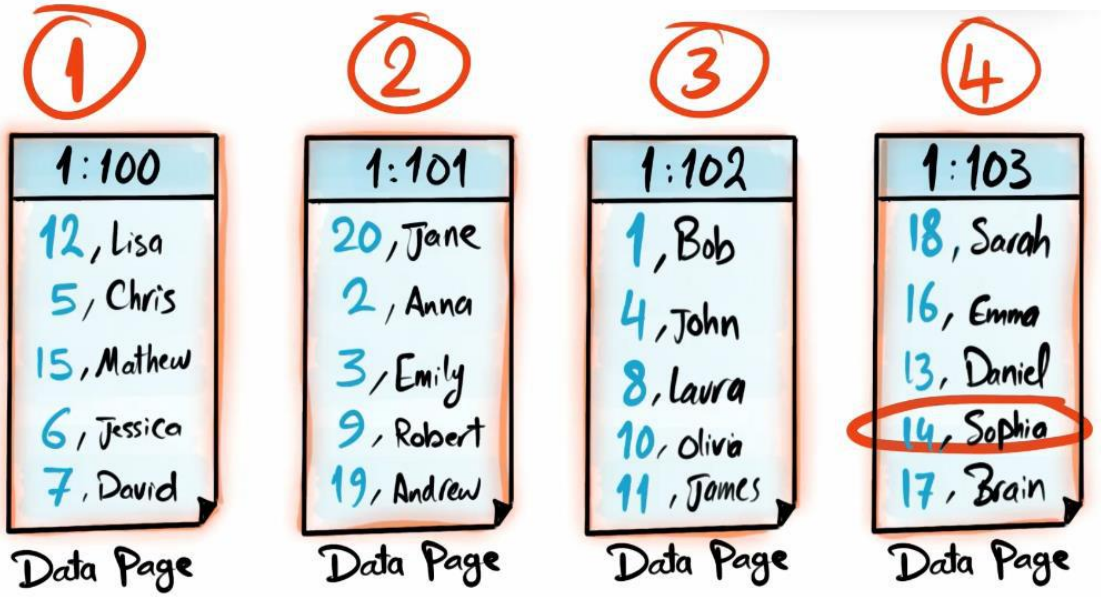

Root Node
Intermediate Nodes
Leaf Level
Index Page
1:300
1-10
1 → 1:200
11 → 1:201
11-20
1:200
1-5
1 → 1:100
6 → 1:101
6-10
Index Page
1:201
11-15
11 → 1:102
16 → 1:103
16-20
Index Page
1:100
1, Bob
2, Anna
3, Emily
4, John
5, Chris
Data Page
1:101
6, Jessica
7, David
8, Laura
9, Robert
10, olivia
Data Page
1:102
11, James
12, Lisa
13, Daniel
14, Sophia
15, Mathew
Data Page
1:103
16, Emma
17, Brian
18, Sarah
19, Andrew
20, Jane
Data Page

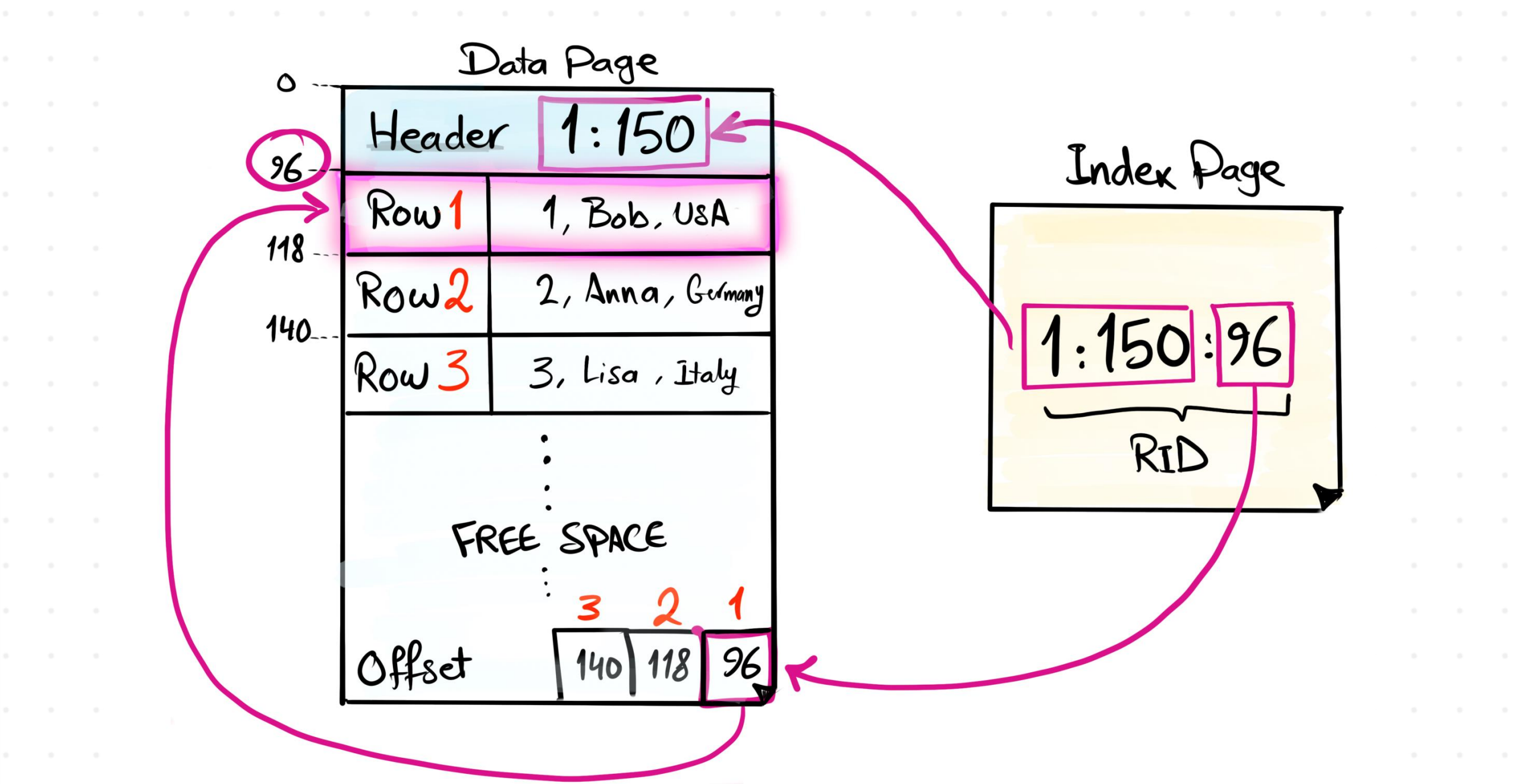

Root Node
Intermediate Nodes
Leaf Level
Index Page
1:300
1 → 1:200
11 → 1:201
1-10
11-20
1
2
3
14?
1:200
1 → 1:100
6 → 1:101
1-5
6-10
Index Page
1:201
11 → 1:102
16 → 1:103
11-15
16-20
Index Page
1:100
1, Bob
2, Anna
3, Emily
4, John
5, Chris
Data Page
1:101
6, Jessica
7, David
8, Laura
9, Robert
10, olivia
Data Page
1:102
11, James
12, Lisa
13, Daniel
14, Sophia
15, Mathew
Data Page
1:103
16, Emma
17, Brian
18, Sarah
19, Andrew
20, Jane
Data Page

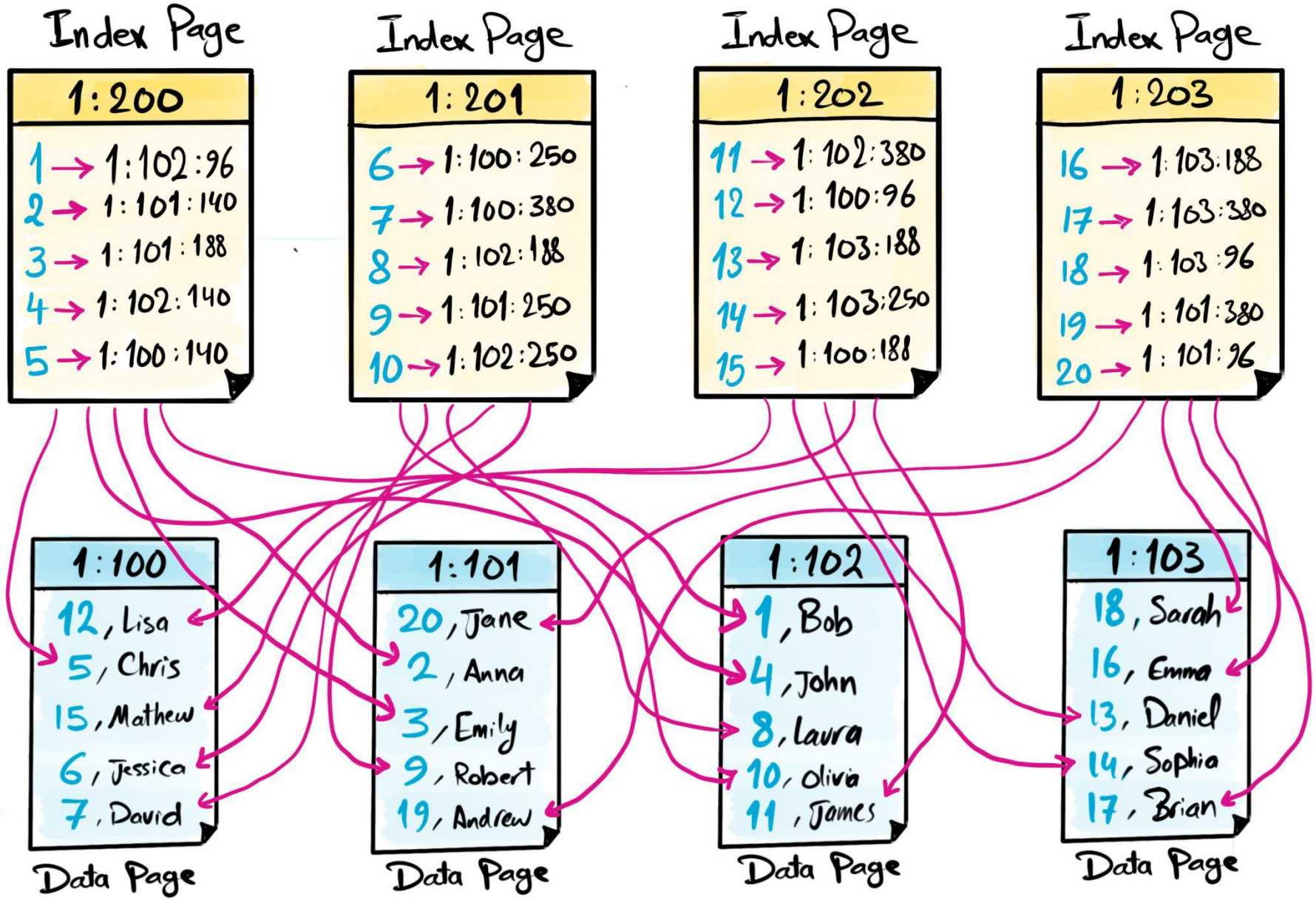
Clustered INDEX

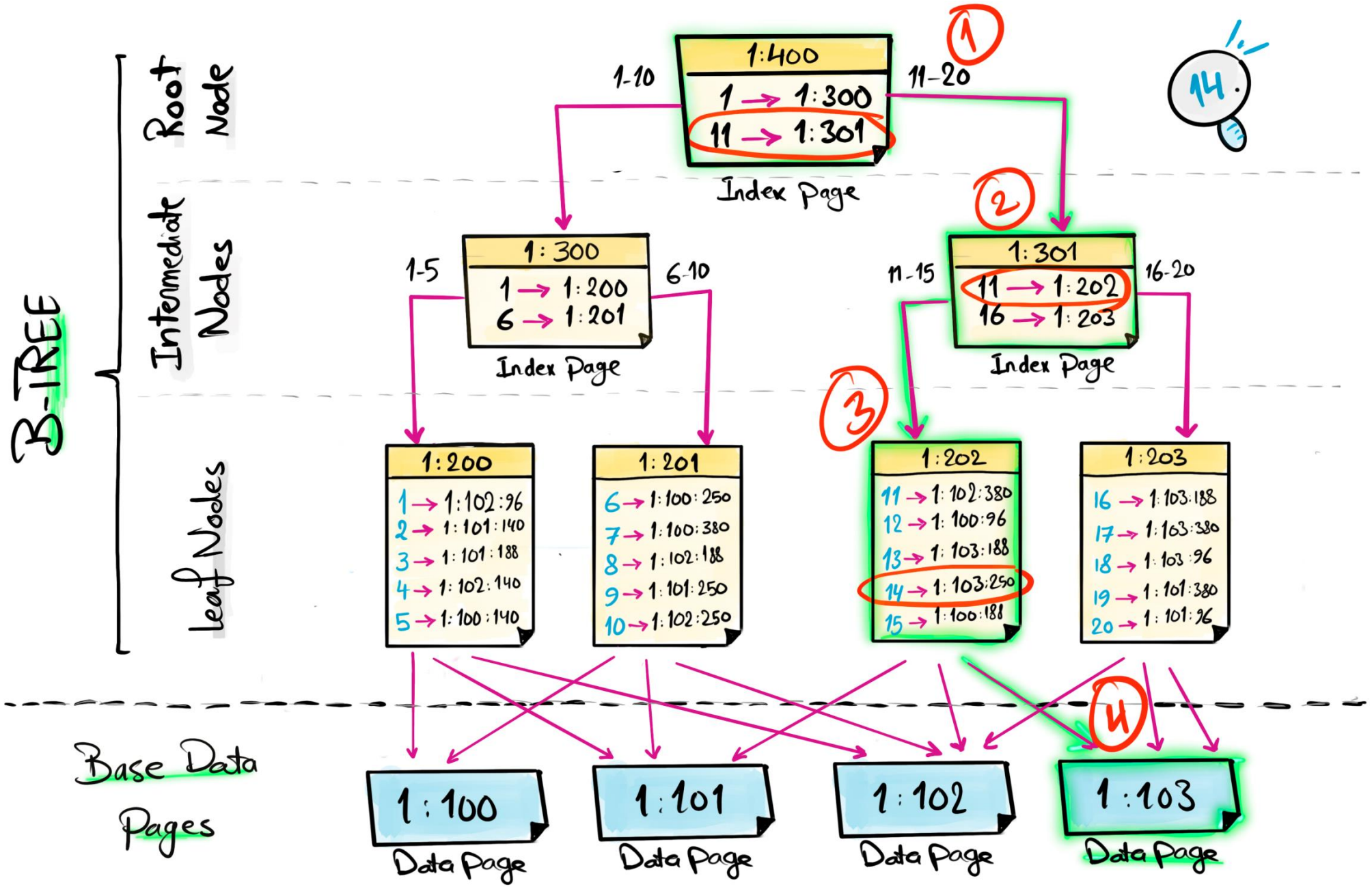


Heap structure



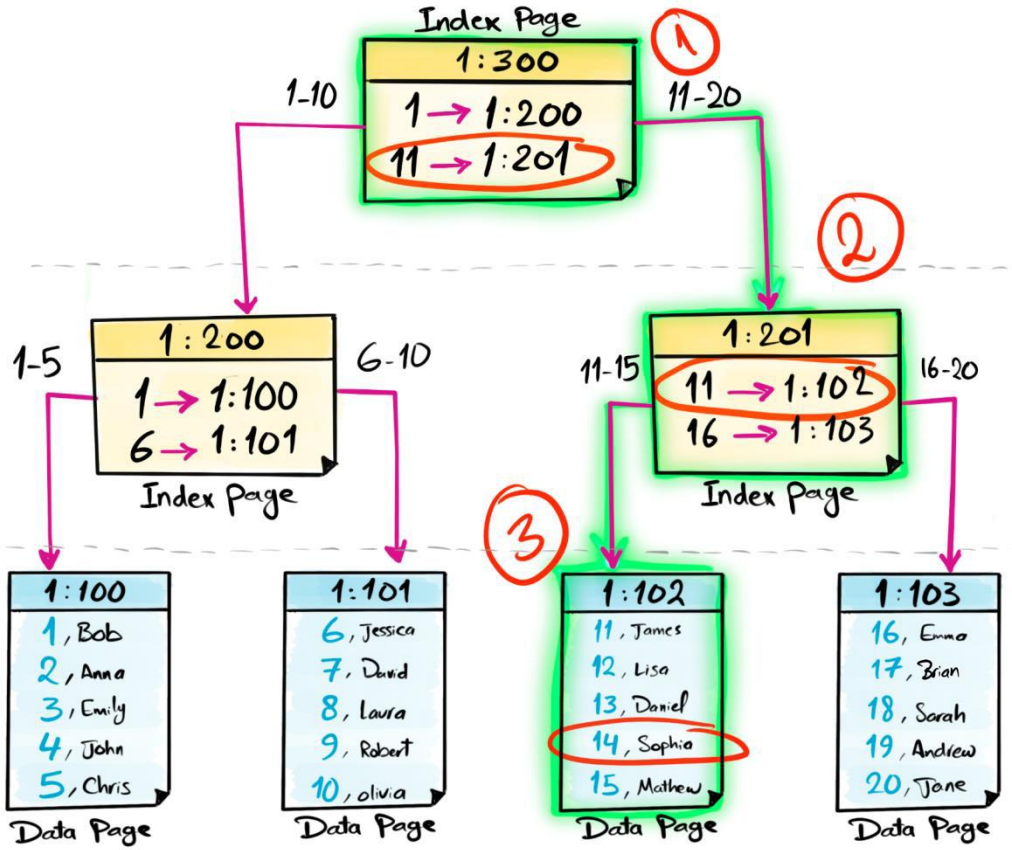

Data Page
0
Header
1:150
96
Row 1
1, Bob, USA
118
Row 2
2, Anna, Germany
140
Row 3
3, Lisa , Italy
FREE SPACE
3
2
1
Offset
140
118
96
Index Page
1:150
96
RID


Index Page
1:200
1 → 1:102:96
2 → 1:101:140
3 → 1:101:188
4 → 1:102:140
5 → 1:100:140
Index Page
1:201
6 → 1:100:250
7 → 1:100:380
8 → 1:102:188
9 → 1:101:250
10 → 1:102:250
Index Page
1:202
11 → 1:102:380
12 → 1:100:96
13 → 1:103:188
14 → 1:103:250
15 → 1:100:188
Index Page
1:203
16 → 1:103:188
17 → 1:103:380
18 → 1:103:96
19 → 1:101:380
20 → 1:101:96
1:100
12, Lisa
5, Chris
15, Mathew
6, Jessica
7, David
Data Page
1:101
20, Jane
2, Anna
3, Emily
9, Robert
19, Andrew
Data Page
1:102
1, Bob
4, John
8, Laura
10, Olivia
11, James
Data Page
1:103
18, Sarah
16, Emma
13, Daniel
14, Sophia
17, Brian
Data Page

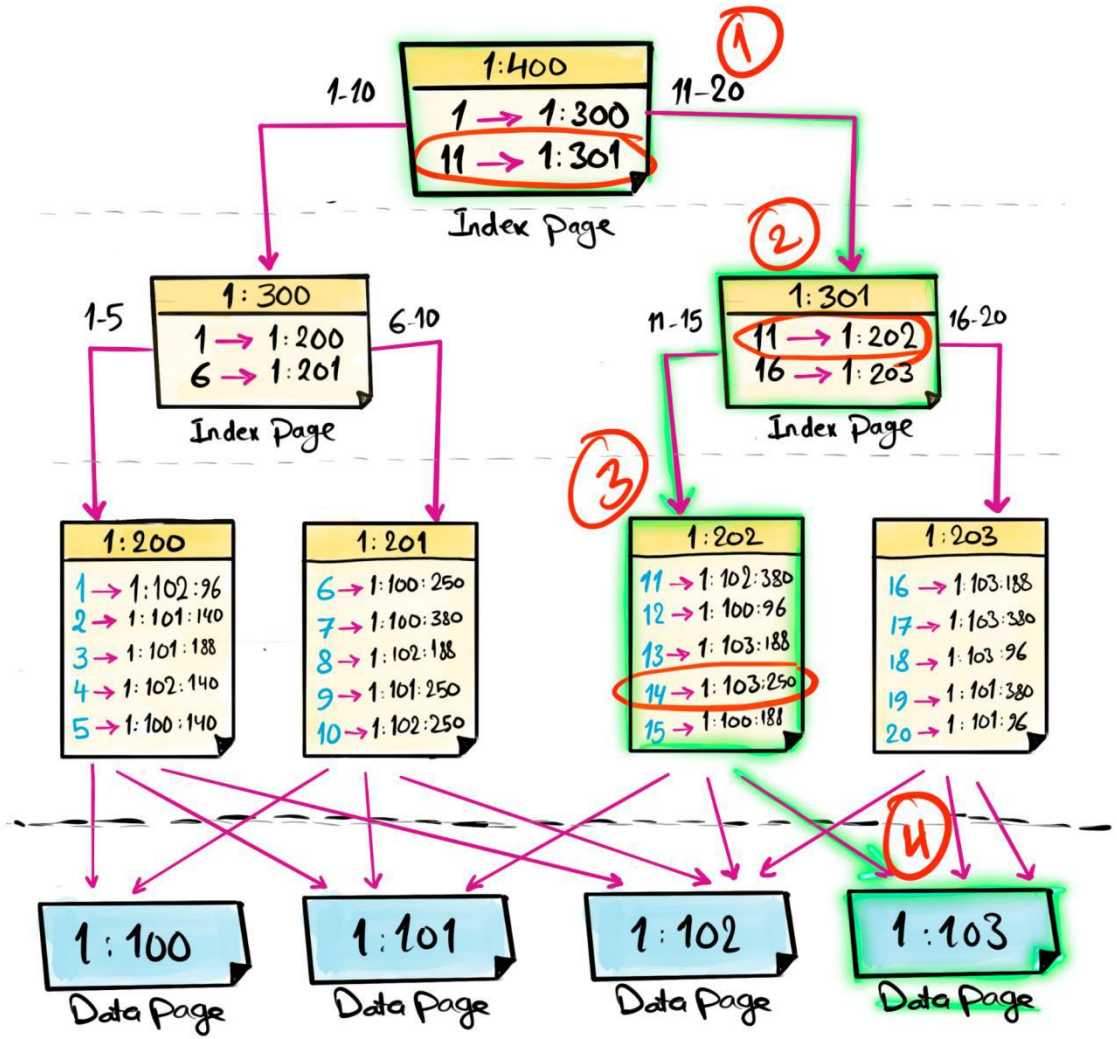

B-TREE
Root Node
Intermediate Nodes
Leaf Nodes
Base Data Pages
1:400
1 → 1:300
11 → 1:301
1-10
11-20
Index Page
1:300
1 → 1:200
6 → 1:201
1-5
6-10
Index Page
1:301
11 → 1:202
16 → 1:203
11-15
16-20
Index Page
1:200
1 → 1:102:96
2 → 1:101:140
3 → 1:101:188
4 → 1:102:140
5 → 1:100:140
1:201
6 → 1:100:250
7 → 1:100:380
8 → 1:102:188
9 → 1:101:250
10 → 1:102:250
1:202
11 → 1:102:380
12 → 1:100:96
13 → 1:103:188
14 → 1:103:250
15 → 1:100:188
1:203
16 → 1:103:188
17 → 1:103:380
18 → 1:103:96
19 → 1:101:380
20 → 1:101:96
1:100
Data Page
1:101
Data Page
1:102
Data Page
1:103
Data Page
1
2
3
4
14

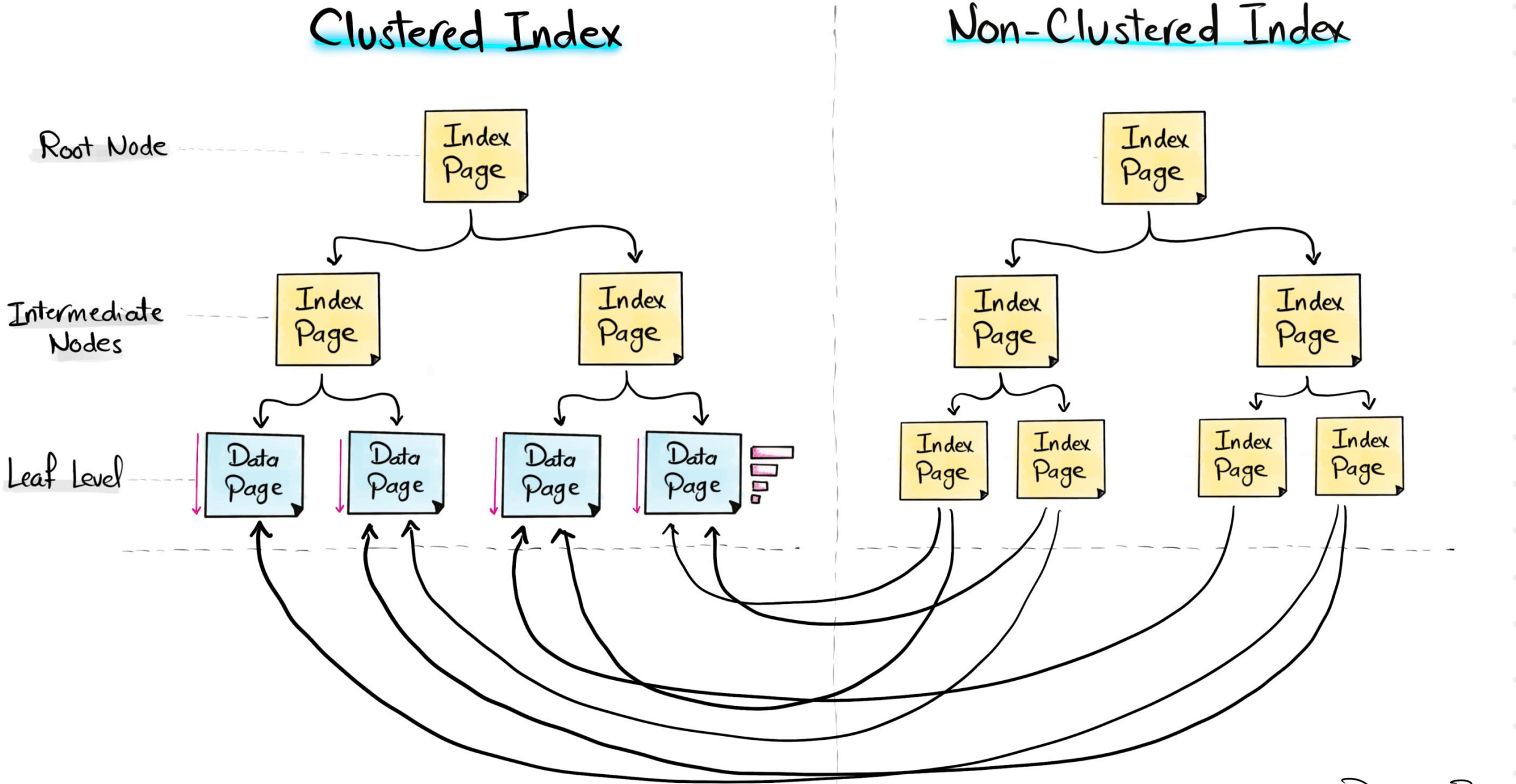
DATA WITH BARAA

Clustered INDEX



Non-Clustered Index





DATA WITH BARAA

DATA WITH BARAA

	Clustered Index	Non-Clustered Index
Definition	**Physically sorts and stores rows**	**Separate structure with pointers to the data**
Numebr of Indexes	**One Index per Table**	**Multiple indexes are allowed**
Read Performance	**Faster**	**Slower**
Write Performance	**Slower**, due to potential data row reordering	**Faster**, since physical data order is unaffected
Storage Efficiency	**More storage-efficient**	**Requires additional storage space**
Use Case	• **Unique Column** • **Not frequently modified Column** • **Improve range query performance**	• **Columns frequently used in search conditions and joins** • **Exact match queries**

Index Syntax

DATA WITH BARAA

Default is NONCLUSTERED

```
CREATE [CLUSTERED | NONCLUSTERED] INDEX index_name ON table_name (column1, column2, ...)
```

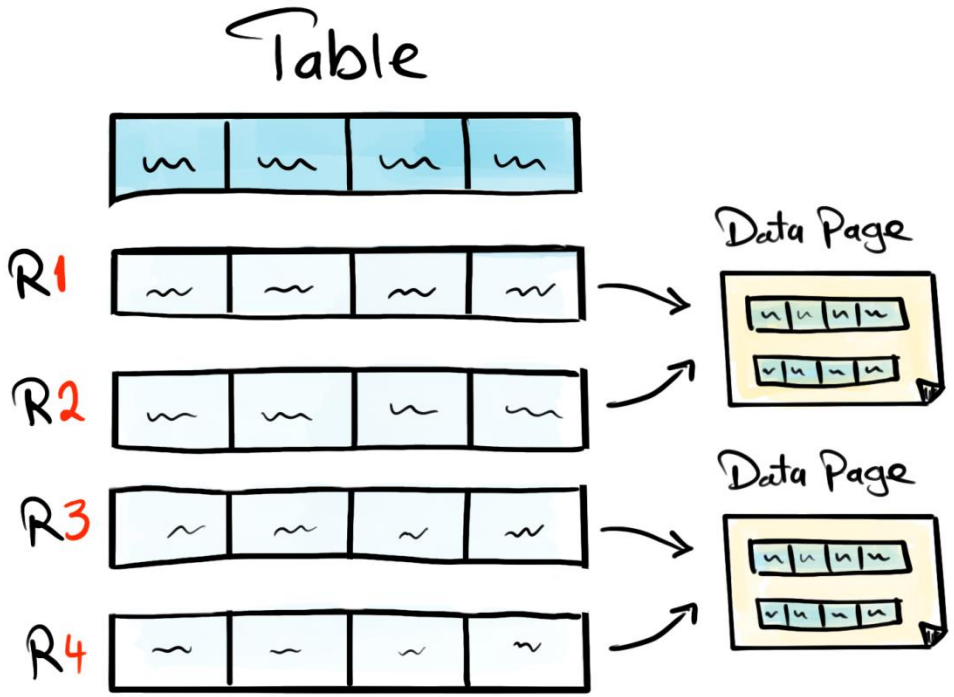
```
CREATE CLUSTERED INDEX IX_Customers_ID ON Customers (ID)
```

```
CREATE NONCLUSTERED INDEX IX_Customers_City ON Customers (City)
```

```
CREATE INDEX IX_Customers_Name ON Customers (LastName ASC, FirstName DESC)
```

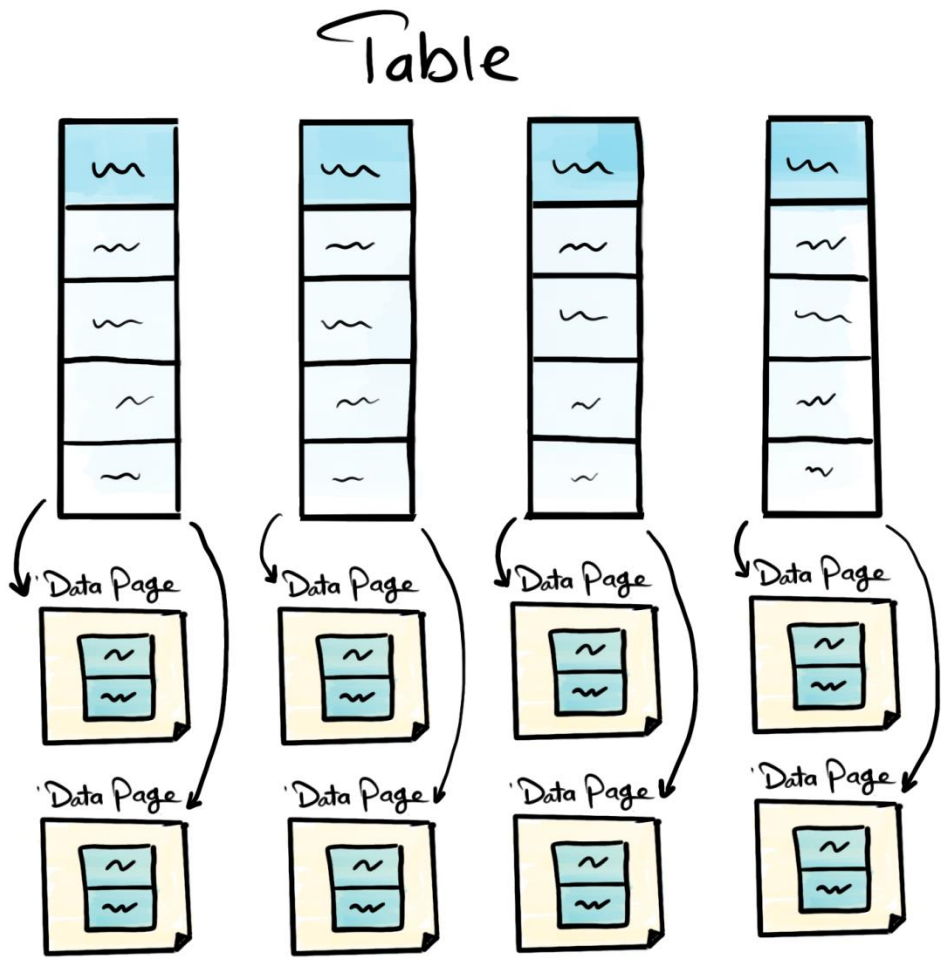
NONCLUSTERED

Rowstore Index

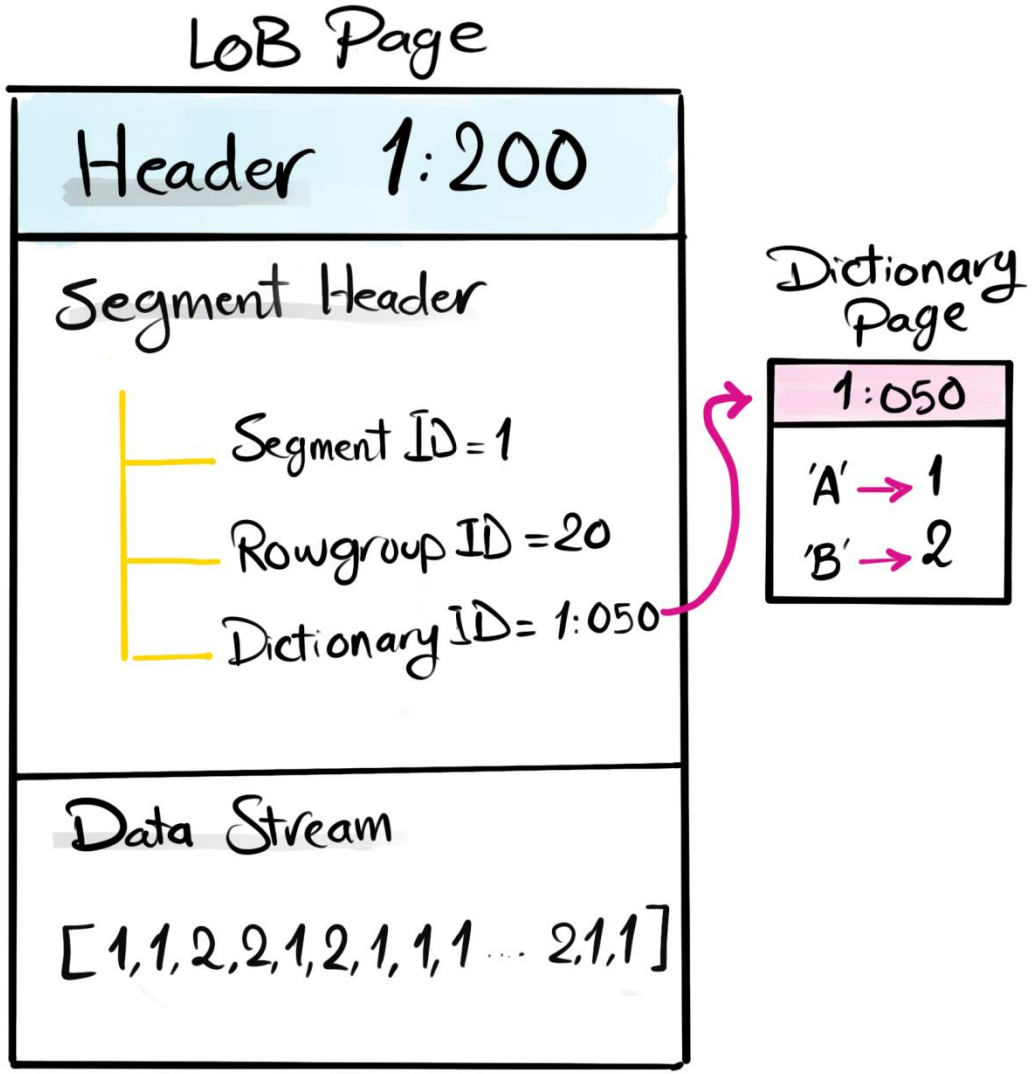


Columnstore Index

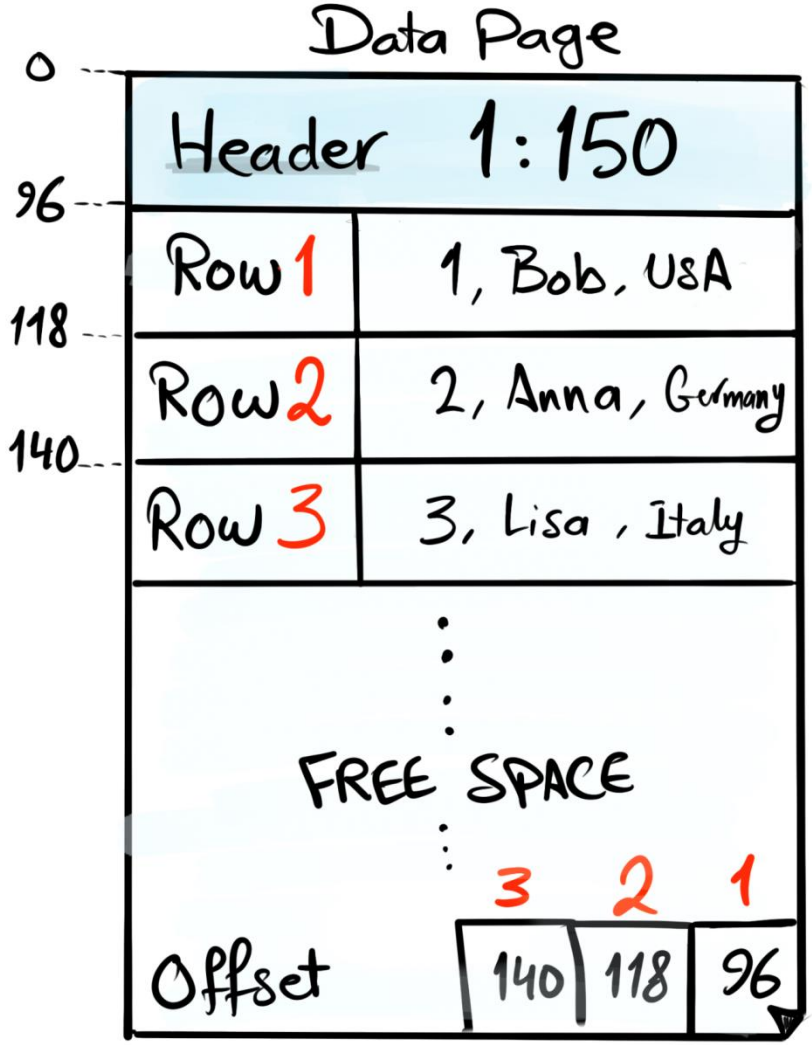
DATA WITH BARAA



Columnstore

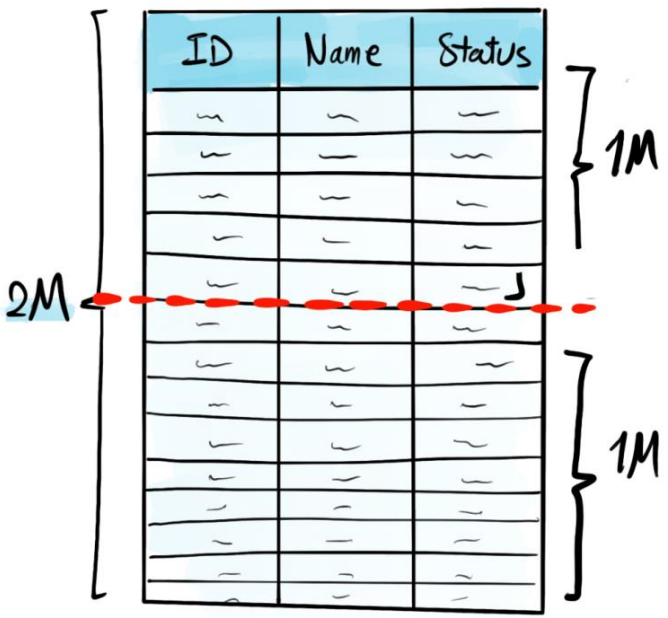


Rowstore

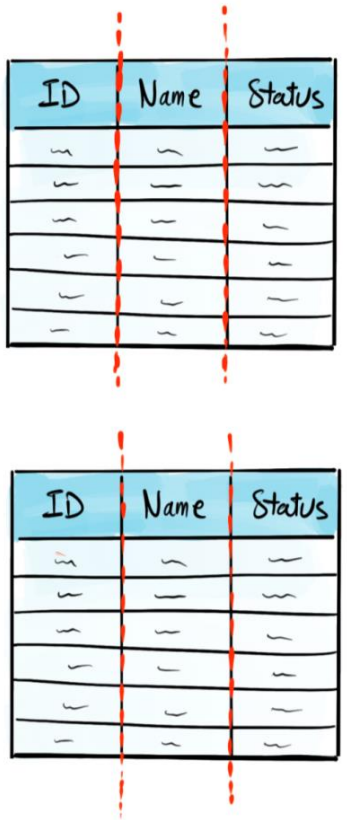


Columnstore Process

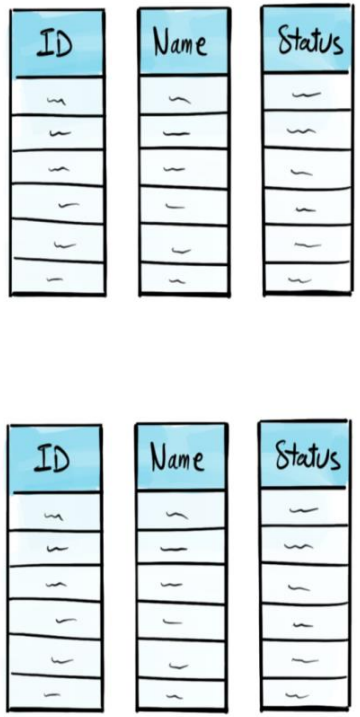
Rowstore Heap Table



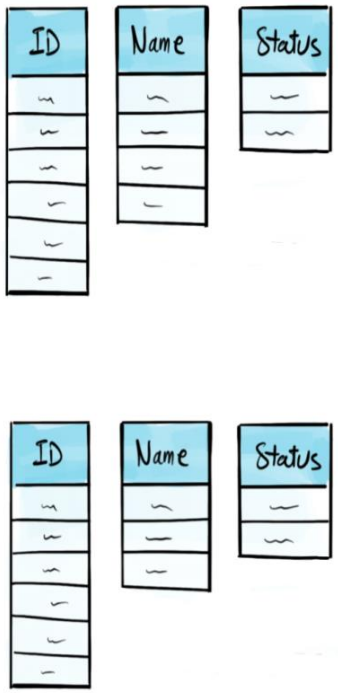
#1 Row Groups



#2 Column Segment

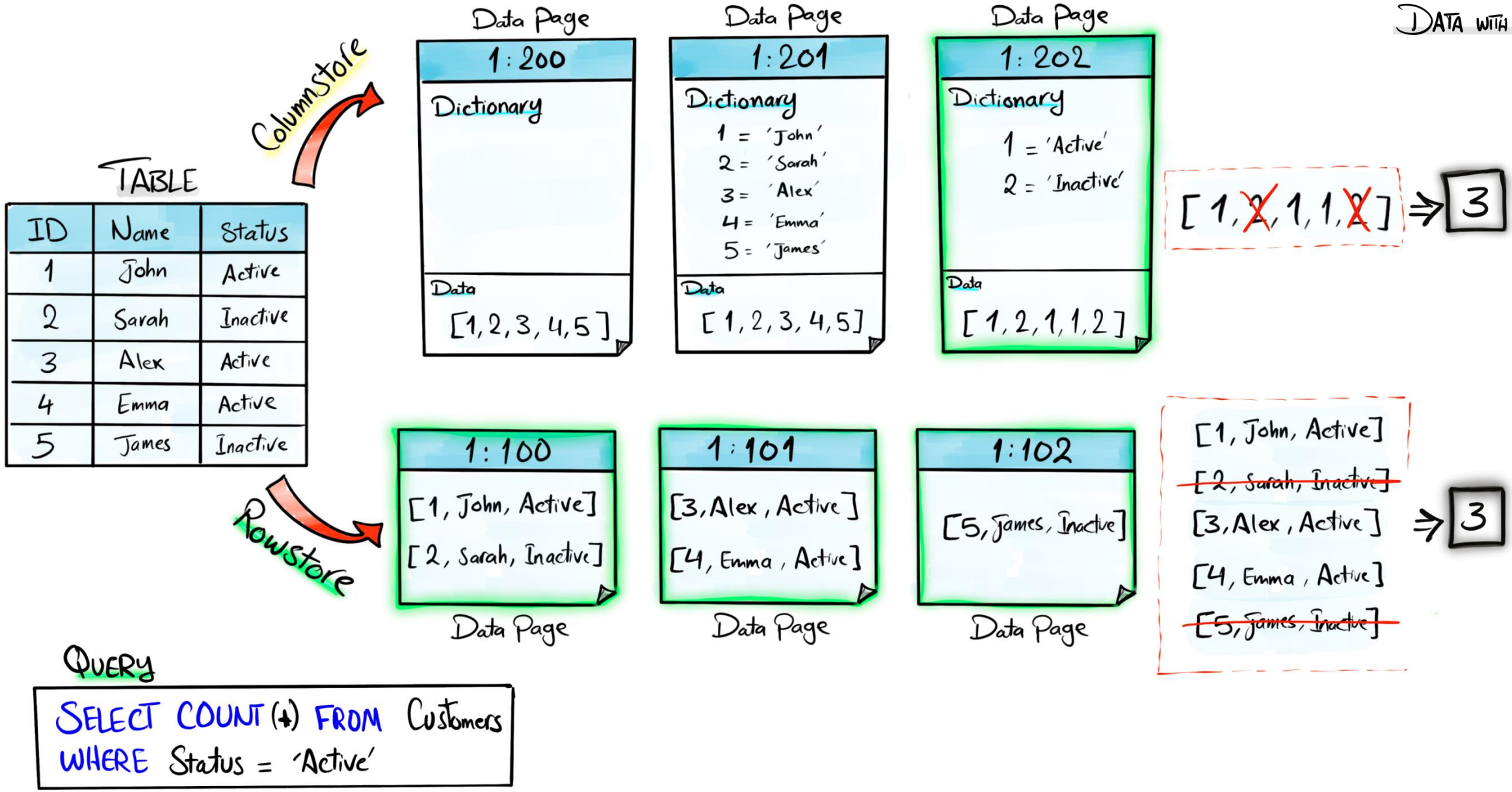


#3 Compression



#4 Store




Columnstore
TABLE
ID | Name | Status
1 | John | Active
2 | Sarah | Inactive
3 | Alex | Active
4 | Emma | Active
5 | James | Inactive
Rowstore
Data Page
1:200
Dictionary
Data
[1,2,3,4,5]
Data Page
1:201
Dictionary
1 = 'John'
2 = 'Sarah'
3 = 'Alex'
4 = 'Emma'
5 = 'James'
Data
[1,2,3,4,5]
Data Page
1:202
Dictionary
1 = 'Active'
2 = 'Inactive'
Data
[1,2,1,1,2]
[1, ~~2~~, 1, 1, ~~2~~] ⇒ 3
Data Page
1:100
[1, John, Active]
[2, Sarah, Inactive]
Data Page
1:101
[3, Alex, Active]
[4, Emma, Active]
Data Page
1:102
[5, James, Inactive]
Data Page
[1, John, Active]
~~[2, Sarah, Inactive]~~
[3, Alex, Active]
[4, Emma, Active]
~~[5, James, Inactive]~~
⇒ 3
QUERY
SELECT COUNT(*) FROM Customers
WHERE Status = 'Active'

DATA WITH BARAA

	Rowstore Index	Columnstore Index
Definition	Organizes and stores data **row by row**	Organizes and stores data **column by column**
Storage Efficiency	**Less efficient in storage**	**Highly efficient with Compression**
Read/Write Optimization	**Fair speed for read & write operations**	**Fast read performance** **Slow write performance**
I/O Efficiency	**Lower (retrieves all columns)**	**Higher (retrieves specific columns)**
Best for ..	**OLTP (Transactional)** commerce, banking, Financial systems, order processing	**OLAP (Analytical)** Data Warehouse, Business intelligence, Reporting, Analytics
Use Case	**- High-frequency transaction applications** **- Quick access to complete records**	**- Big Data Analytics** **- Scanning of large datasets** **- Fast aggregation**

Columnstore Index

DATA WITH BARAA

Default is ROWSTORE

CRE

e

Unique Index

Rowstore

Ensures no duplicate values exist in specific column.

ntry)

Benefits

Columnstore

1. Enforce uniqueness
2. Slightly increase query perfromance

stomers (Country)

NOT ALLOWED TO USE COLUMNS

Rules

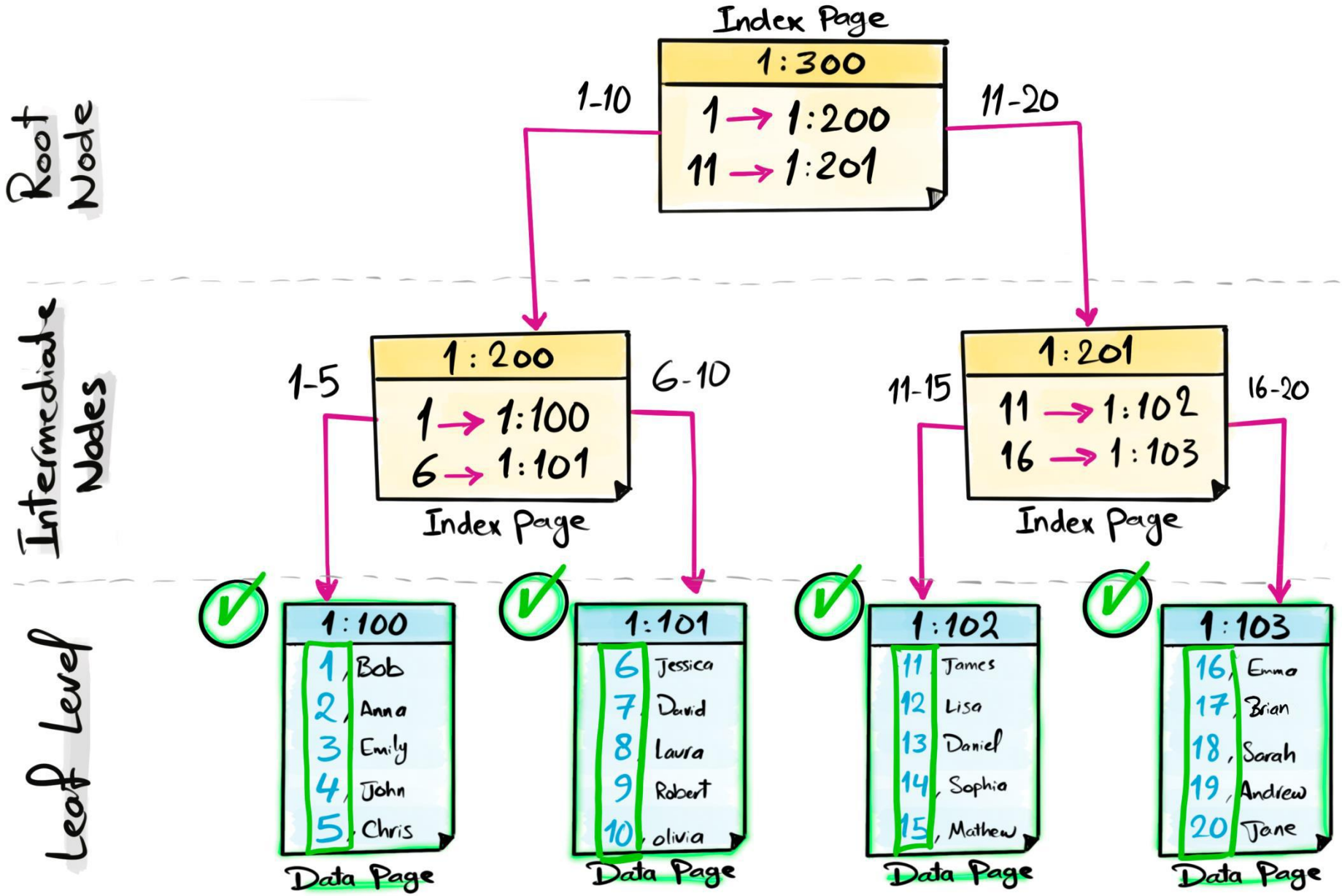
- You **can't** specific columns in **Clustered** Index Columnstore

Unique Index

Ensures no duplicate values exist in specific column.

Benefits

1. Enforce uniqueness
2. Slightly increase query perfromance


Root Node
Intermediate Nodes
Leaf Level
Index Page
1:300
1-10
1 → 1:200
11 → 1:201
11-20
1:200
1-5
1 → 1:100
6 → 1:101
6-10
Index Page
1:201
11-15
11 → 1:102
16 → 1:103
16-20
Index Page
1:100
1, Bob
2, Anna
3, Emily
4, John
5, Chris
Data Page
1:101
6, Jessica
7, David
8, Laura
9, Robert
10, olivia
Data Page
1:102
11, James
12, Lisa
13, Daniel
14, Sophia
15, Mathew
Data Page
1:103
16, Emma
17, Brian
18, Sarah
19, Andrew
20, Jane
Data Page

Unique Index

Default is NOT Unique

```
CREATE [UNIQUE] [CLUSTERED | NONCLUSTERED] [COLUMNSTORE] INDEX index_name
ON table_name (column1, column2, ...)
```

Index Allows Duplicates

```
CREATE INDEX IX_Customers_Email ON Customers (Email)
```

Duplicates are not allowed

```
CREATE UNIQUE INDEX IX_Customers_Email ON Customers (Email)
```

Filtered Index

An index that includes only rows meeting the specified conditions

Benefits

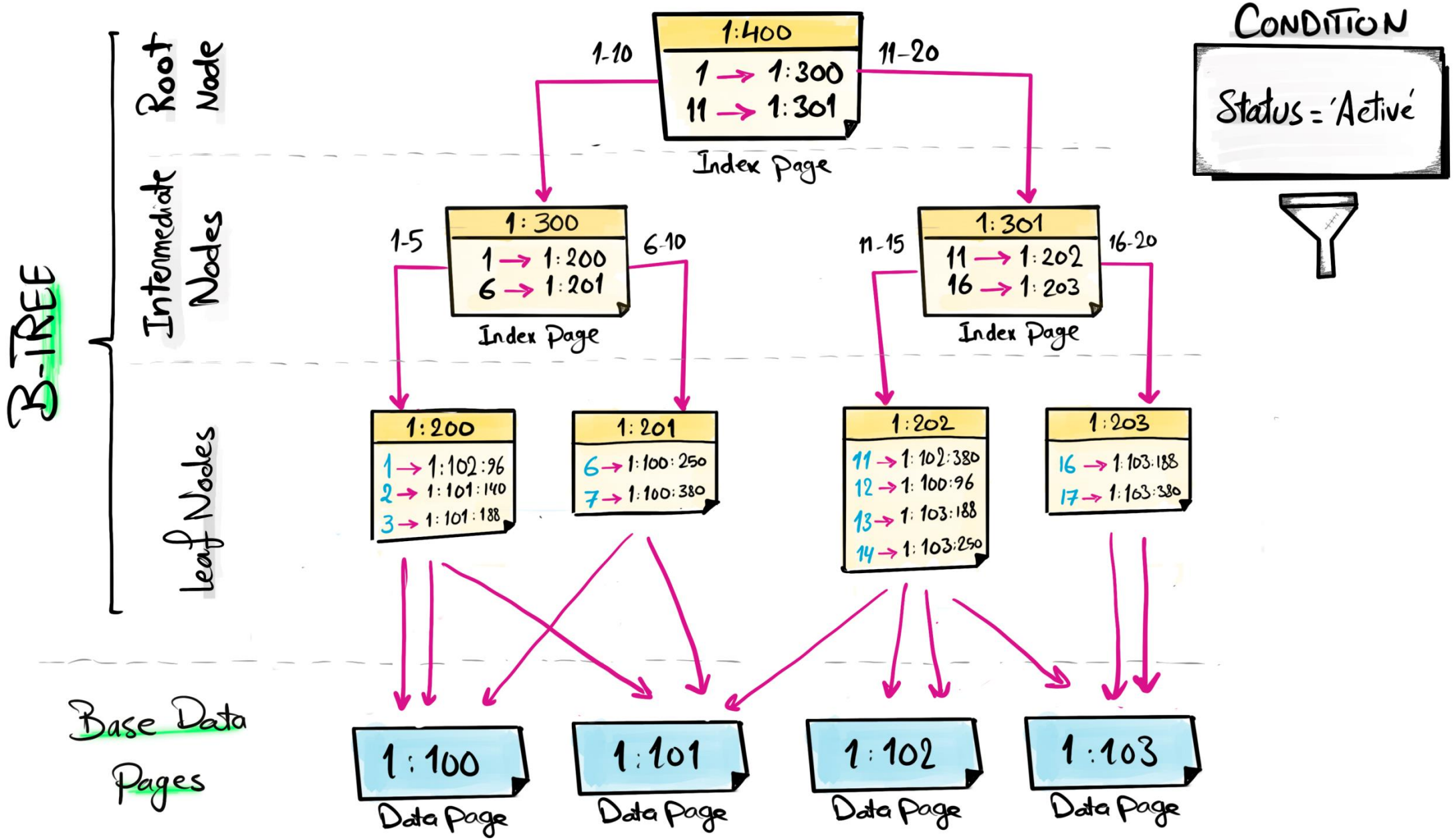
- Targeted Optimization
- Reduce storage: Less data in the index

Filtered Index


```
CREATE [UNIQUE] [NONCLUSTERED] INDEX index_name
ON table_name (column1, column2, ...)
WHERE [Condition]
```

Rules

- You cannot create a filtered index on a clustered index.
- You cannot create a filtered index on a columnstore index.


B-TREE
Root Node
Intermediate Nodes
Leaf Nodes
Base Data Pages
1:400
1 → 1:300
11 → 1:301
Index Page
1-10
11-20
1:300
1 → 1:200
6 → 1:201
Index Page
1-5
6-10
1:301
11 → 1:202
16 → 1:203
Index Page
11-15
16-20
1:200
1 → 1:102:96
2 → 1:101:140
3 → 1:101:188
1:201
6 → 1:100:250
7 → 1:100:380
1:202
11 → 1:102:380
12 → 1:100:96
13 → 1:103:188
14 → 1:103:250
1:203
16 → 1:103:188
17 → 1:103:380
1:100
Data Page
1:101
Data Page
1:102
Data Page
1:103
Data Page
CONDITION
Status = 'Active'

When To Use

Data with Baraa

HEAP

Fast **Inserts**
(For Staging Tables)

Clustered Index

MAIN

OLTP

For **Primary keys**
If not, then for date columns

Columnstore Index

OLAP

For **Analytical** Queries
Reduce Size of Large Table

Non-Clustered Index

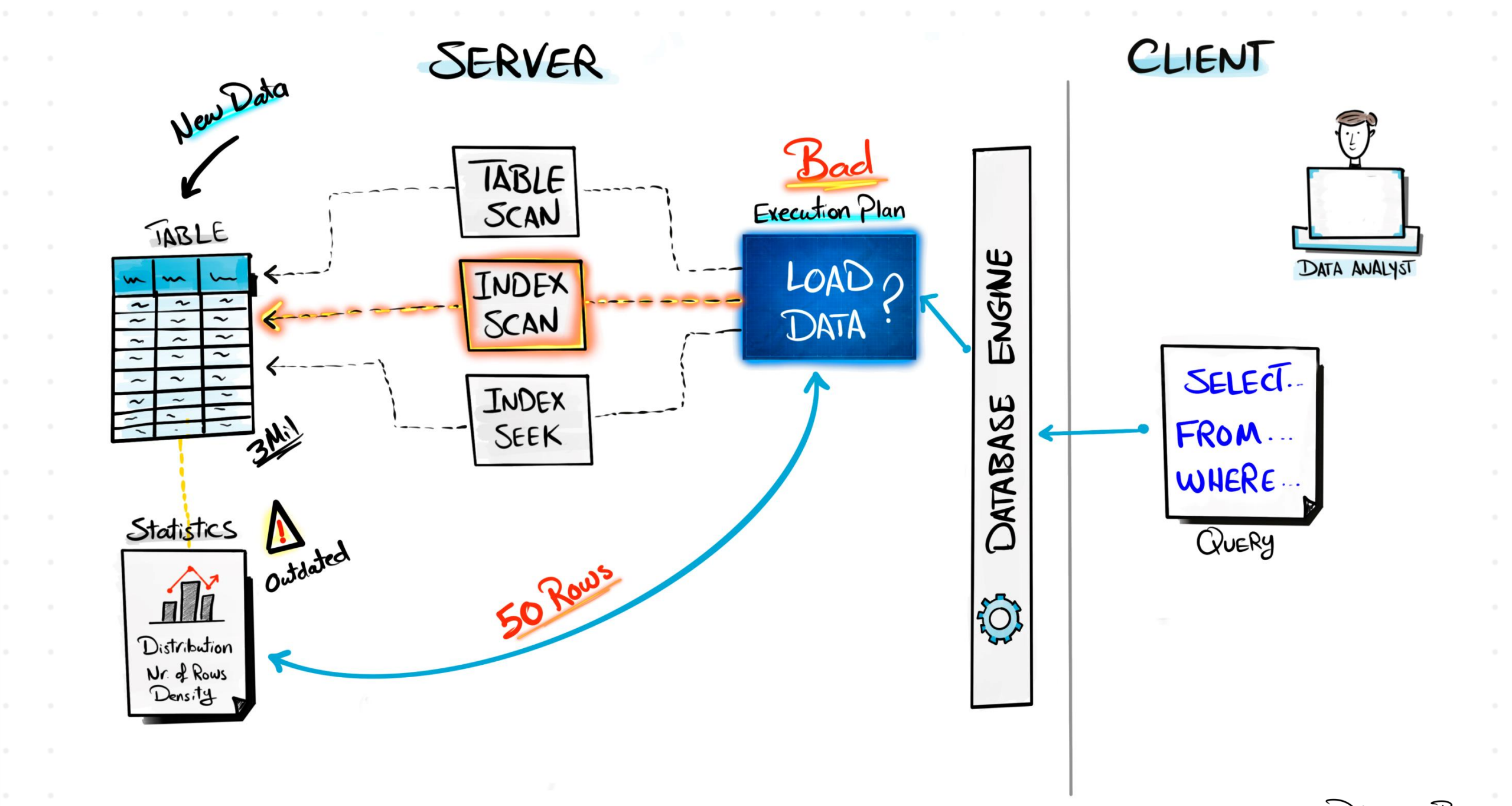
For **non-PK** columns
(Foreign keys, Joins, and Filters)

Filetered Index

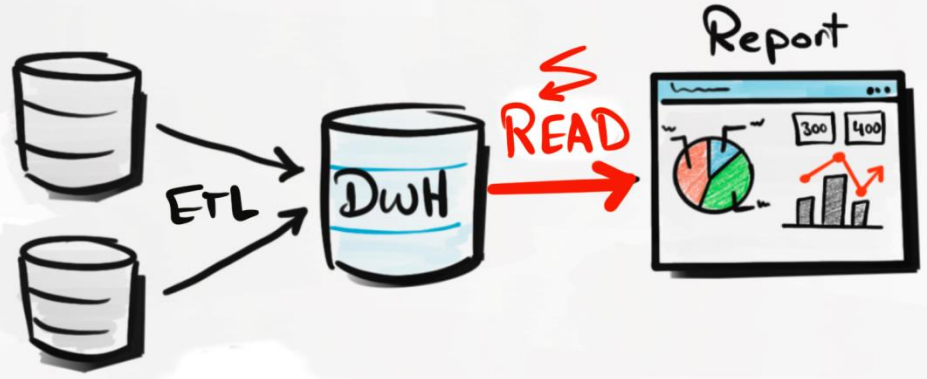
Target **Subset** of Data
Reduce Size of Index

Unique Index

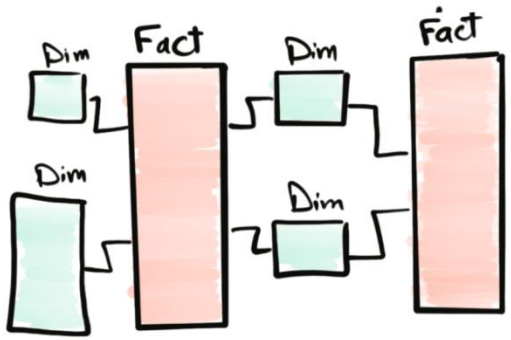
Enforce **Uniqueness**
Improve Query Speed


SERVER
CLIENT
New Data
TABLE
3Mil
TABLE SCAN
INDEX SCAN
INDEX SEEK
Bad
Execution Plan
LOAD DATA ?
DATABASE ENGINE
DATA ANALYST
SELECT...
FROM...
WHERE...
QUERY
Statistics
Outdated
Distribution
Nr. of Rows
Density
50 Rows

OLAP (Analytical)



COLUMNSTORE INDEX



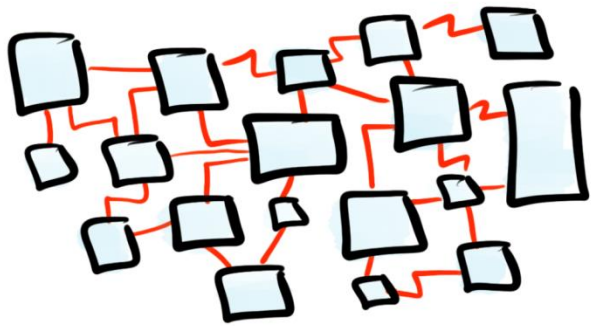
GOAL Optimize **READ** Performance

OLTP (Transaction)

DATA WITH BARAA



CLUSTERED INDEX PK



GOAL Optimize **WRITE** Performance

Indexing Strategy

Data with Baraa

#1 Initial Indexing Strategy

OLAP

Optimize
Read
Performance

Switch **Large** frequently used tables into **ColumnStore**

OLTP

Optimize
Write
Performance

Clustered Index
Primary Keys

#2 Usage Patterns Indexing

1. Identify **frequently** used **Tables** & **Columns**
2. Choose **Right** Index
3. **Test** Index

#3 Scenario-Based Indexing

1. Identify **Slow** Queries
2. Check **Execution Plan**
3. Choose **Right** Index
4. (Test) **Compare** Execution Plans

#4 Monitoring & Maintenance

1. Monitor Index **Usage**
2. Monitor **Missing** Indexes
3. Monitor **Duplicate** Indexes
4. Update **Statistics**
5. Monitor **Fragmentations**



DATA WITH BARAA

Partitioning

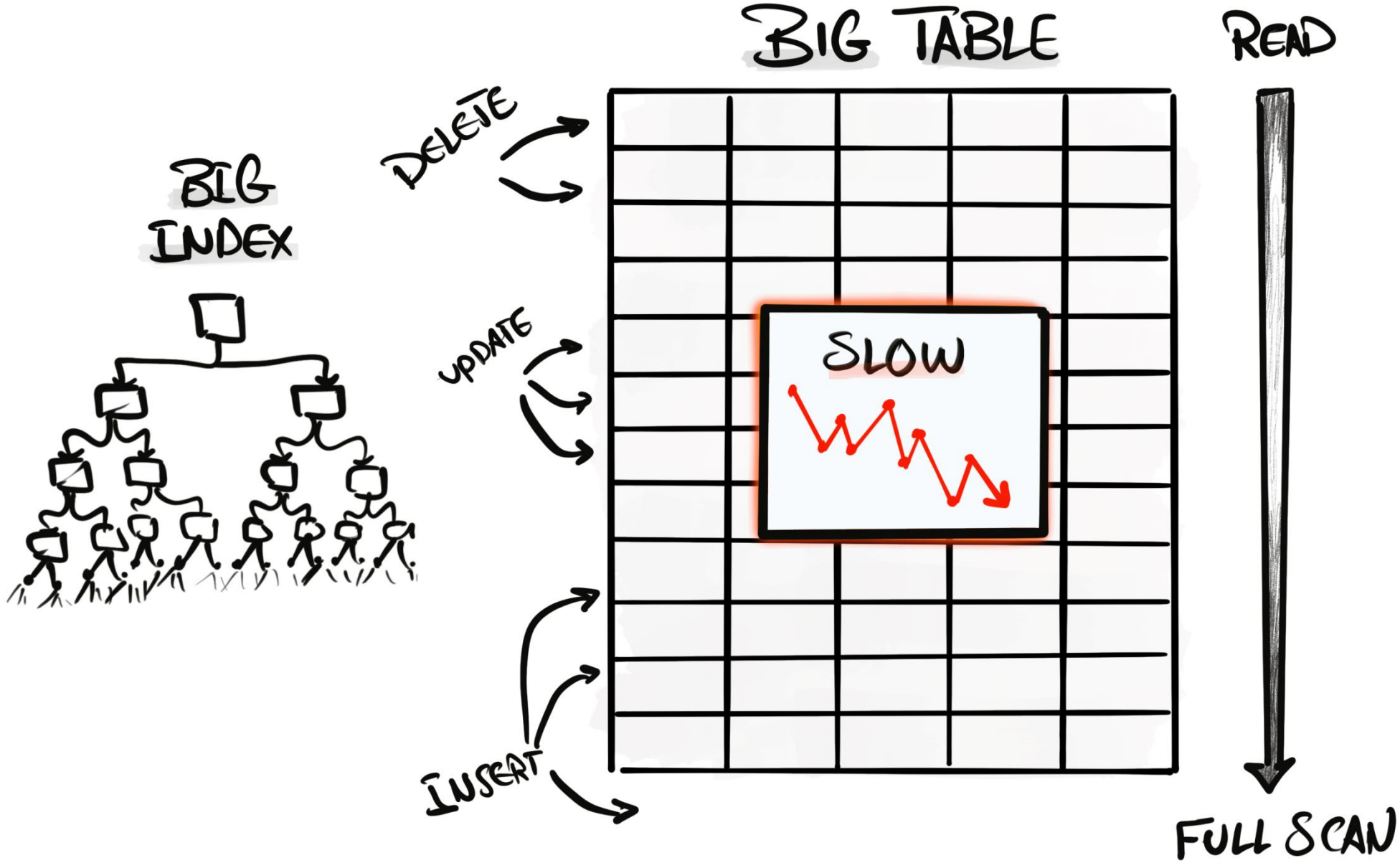
Baraa Khatib Salkini
DATA WITH BARAA
SQL Course | Indexes

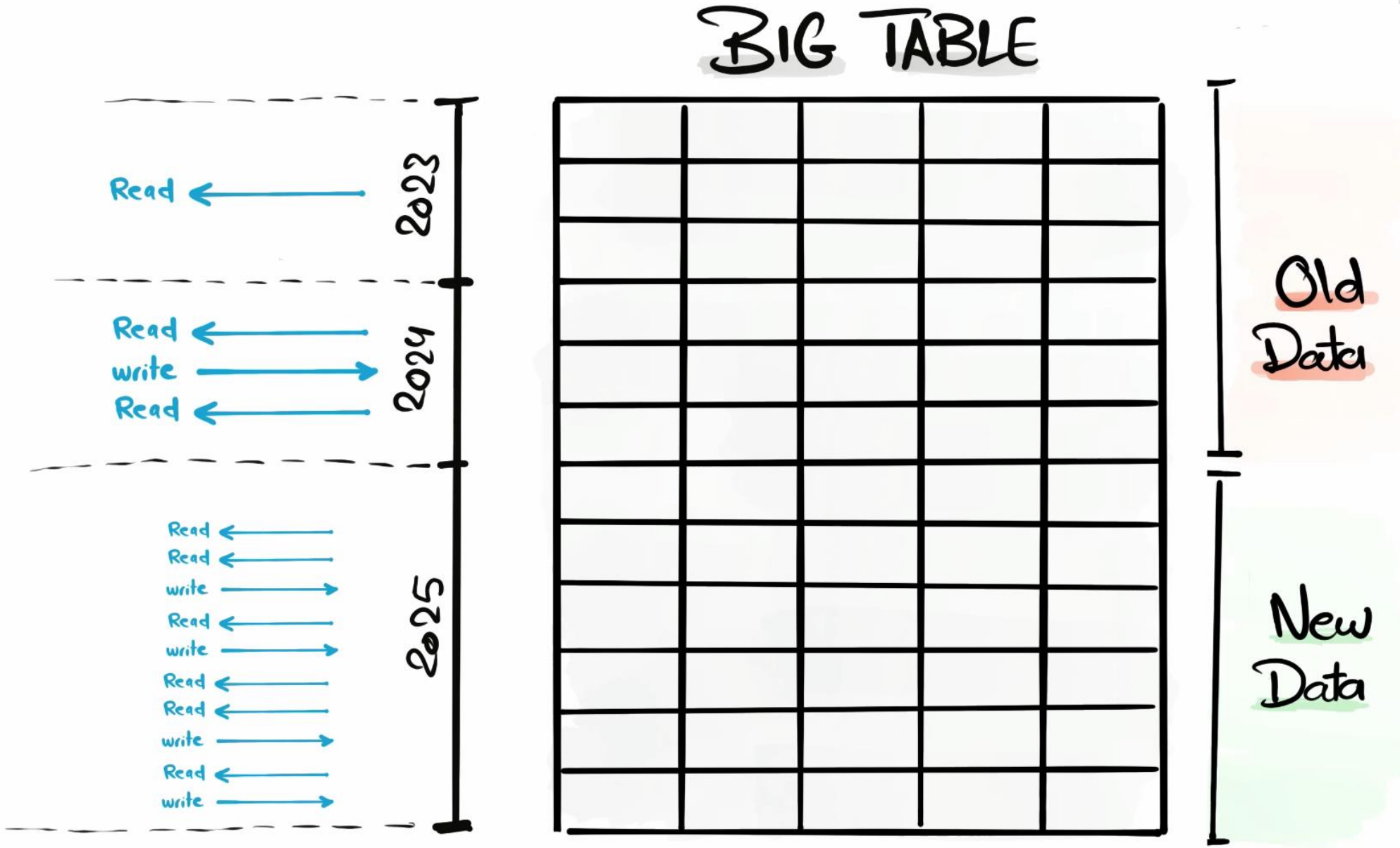


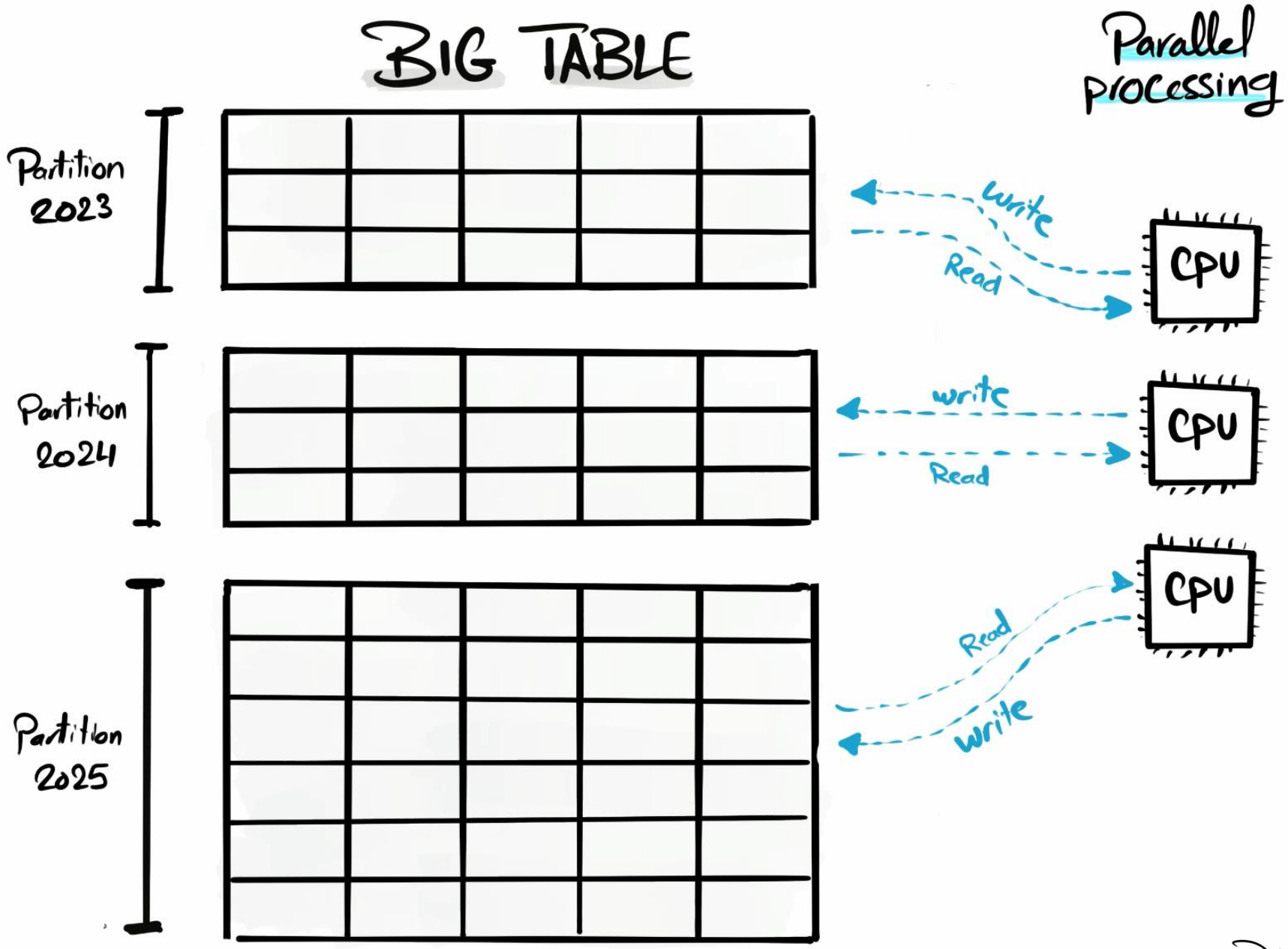
SQL PARTITIONING

Divides Big Table into Smaller Partitions

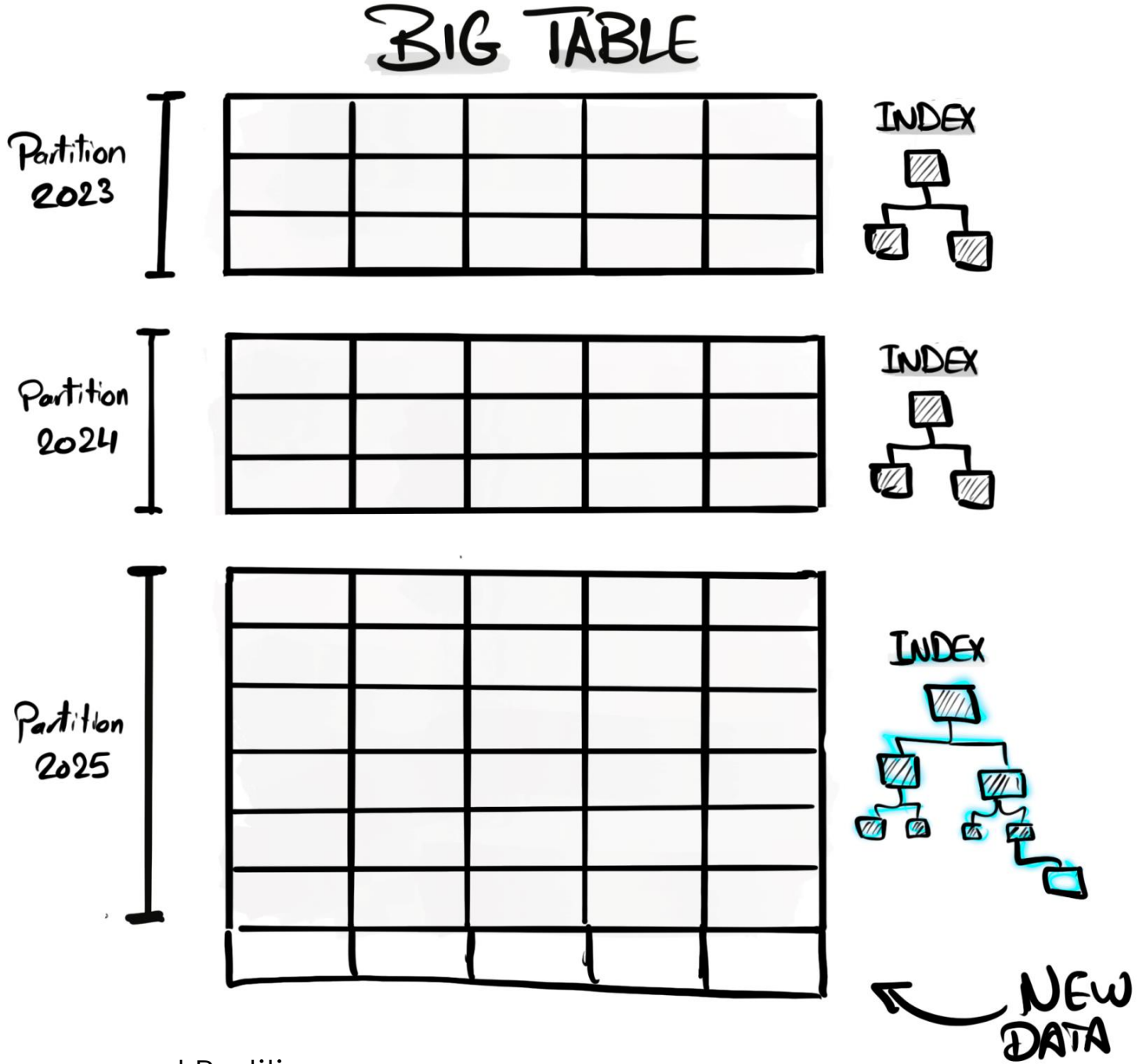
while still being treated as a single logical table.


BIG TABLE
READ
BIG INDEX
DELETE
UPDATE
SLOW
INSERT
FULL SCAN


BIG TABLE
2023
Read
2024
Read
write
Read
2025
Read
Read
write
Read
write
Read
Read
write
Read
write
Old Data
New Data



DATA WITH BARAA


BIG TABLE
Partition 2023
INDEX
Partition 2024
INDEX
Partition 2025
INDEX
NEW DATA

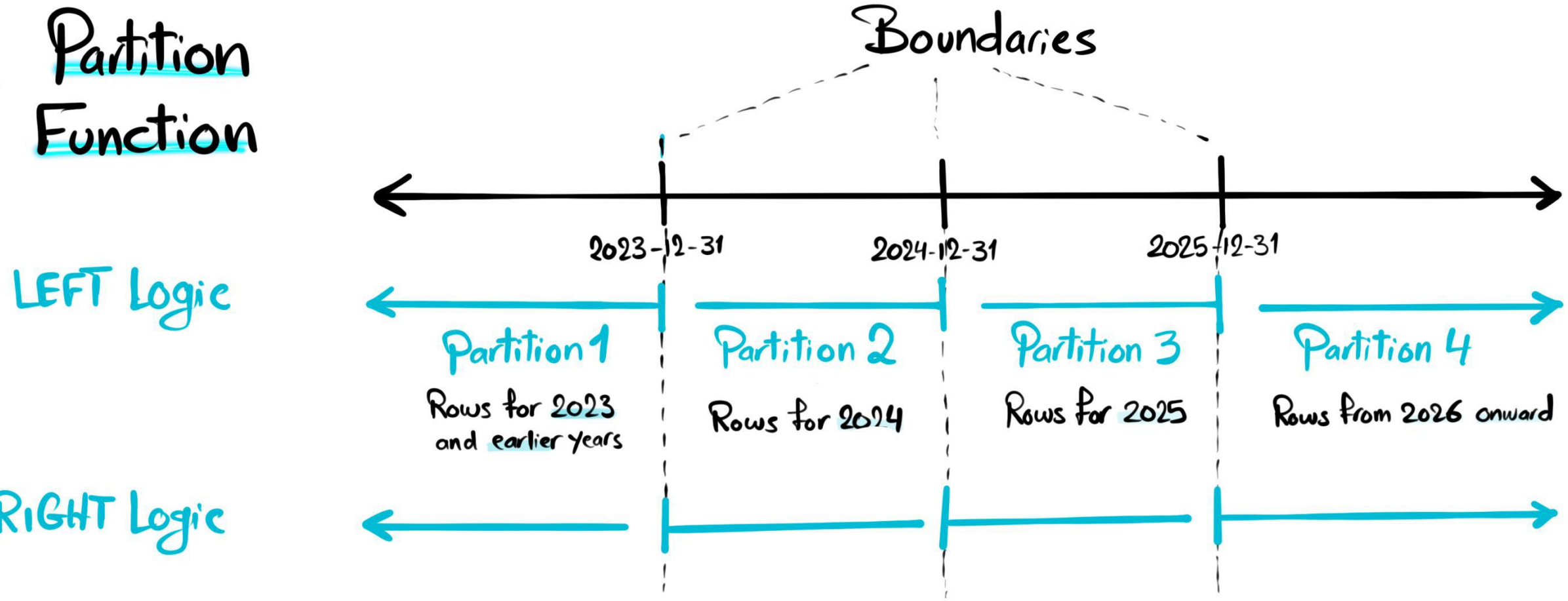
PARTITION FUNCTION

Define the Logic on how to divide

your data into paritions !

Based on Partition Key Like (Column, Region, ..)

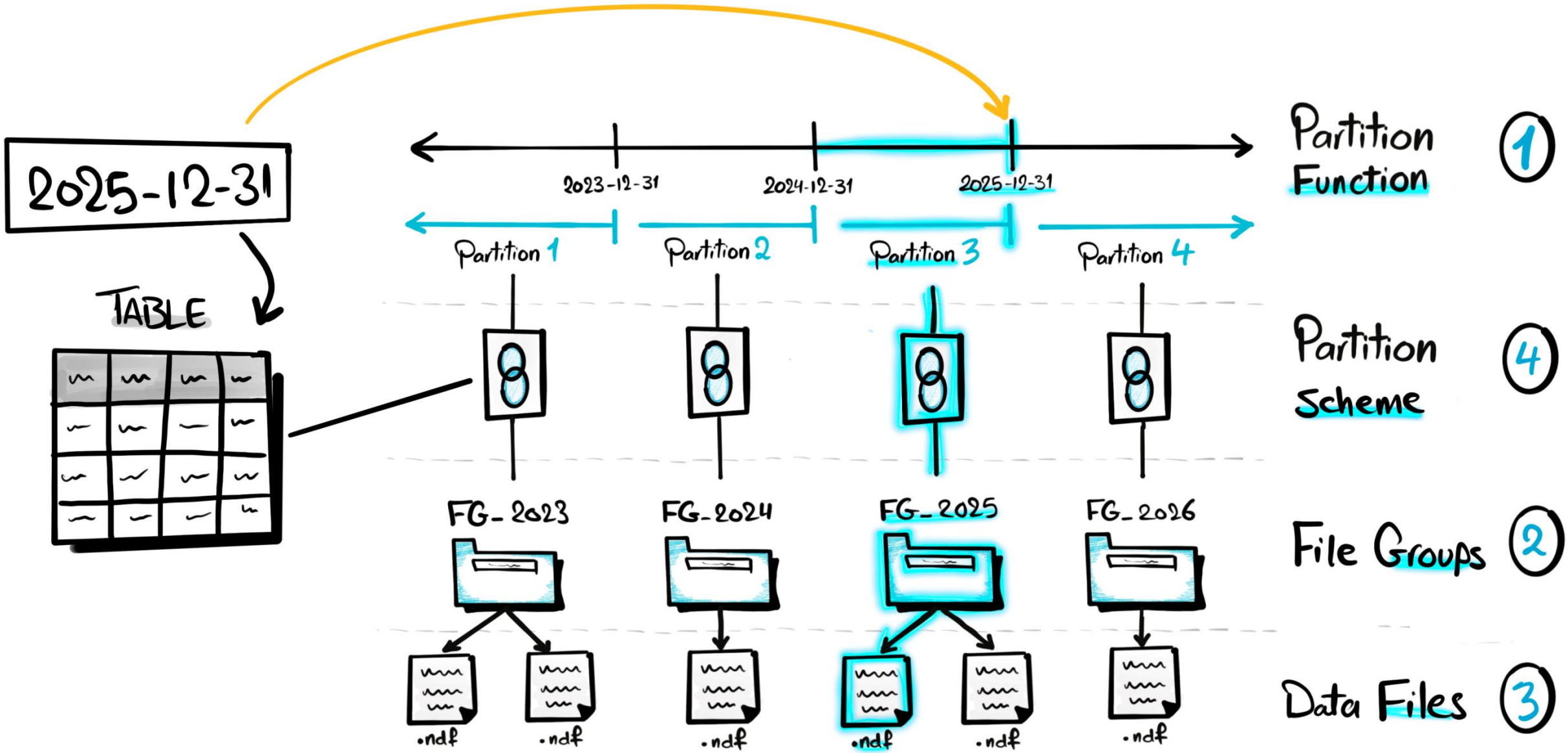
1 Partition Function

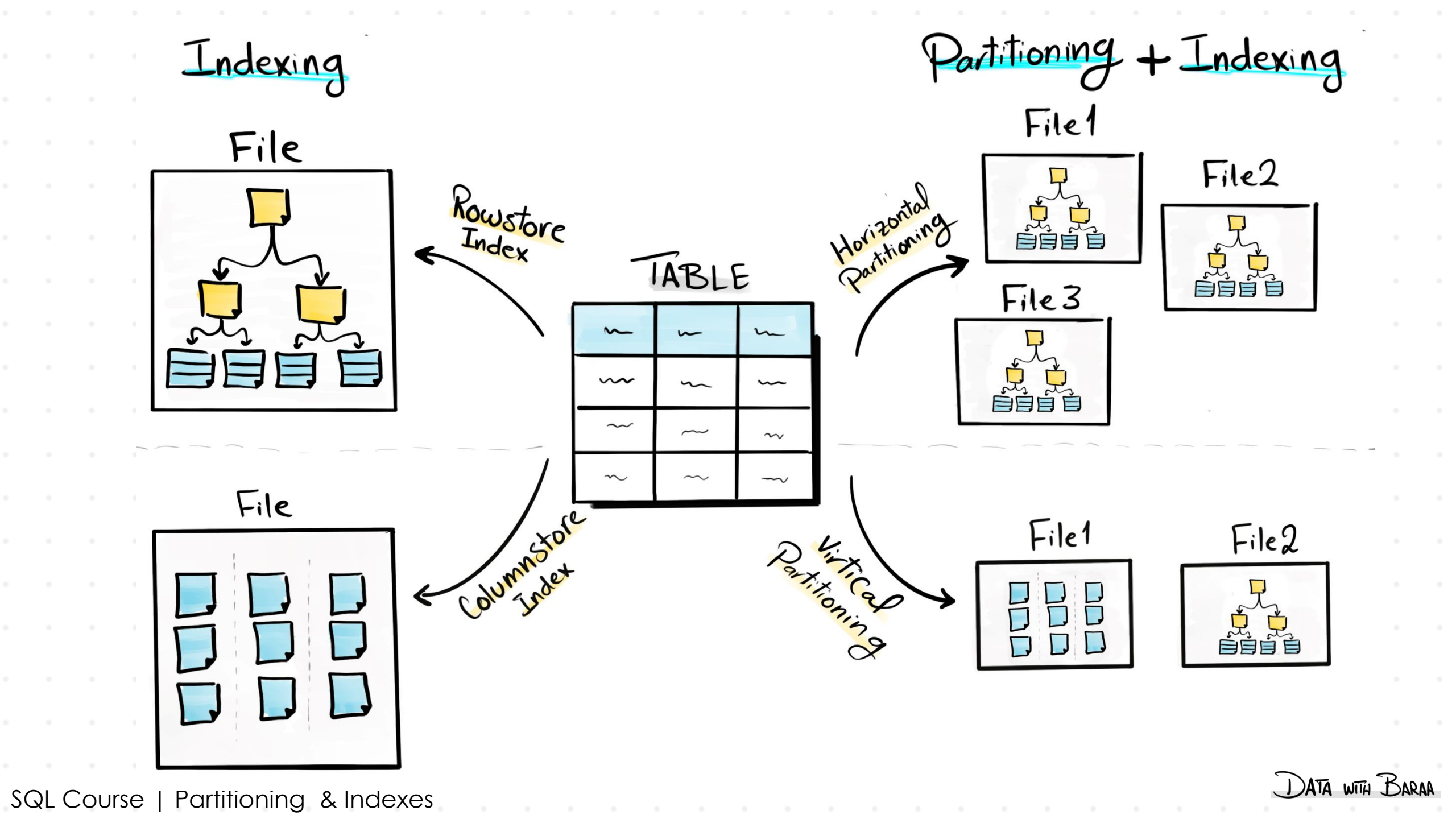


FILEGROUPS

Logical container of one or more data files

to help organize partitions.


2025-12-31
TABLE
2023-12-31
2024-12-31
2025-12-31
Partition 1
Partition 2
Partition 3
Partition 4
Partition Function
1
Partition Scheme
4
FG_2023
FG_2024
FG_2025
FG_2026
File Groups
2
.ndf
.ndf
.ndf
.ndf
.ndf
Data Files
3


Indexing
Partitioning + Indexing
File
Rowstore Index
TABLE
Horizontal Partitioning
File1
File2
File3
File
Columnstore Index
Virtical Partitioning
File1
File2